The PEACE Protocol[1]

A protocol for transferable encryption rights.

Logical Mechanism LLC[2]

April 4, 2026

[1]This project was funded in Fund 14 of Project Catalyst.
[2]Contact: support@logicalmechanism.io

Contents

1 Abstract **1**

2 Introduction **1**

3 Background And Preliminaries **2**
3.1 Parameter Selection . . . 3

4 Cryptographic Primitives Overview **3**
4.1 Register-based . . . 3
4.2 ECIES + AES-GCM . . . 4
4.3 Re-Encryption . . . 5
4.4 Groth16 SNARK Verification . . . 7
4.5 Related Work And Design Rationale . . . 8

5 Protocol Overview **9**
5.1 Design Goals And Requirements . . . 9
5.2 On-Chain And Off-Chain Architecture . . . 10
5.3 Key Management And Identity . . . 11
5.4 Protocol Specification . . . 12

6 Security Model **14**
6.1 Assumptions . . . 14
6.2 Trust Model . . . 15
6.3 Threat Analysis . . . 15
6.4 Metadata Leakage . . . 16
6.5 Limitations And Risks . . . 16
6.6 Performance And On-Chain Cost . . . 18

7 Conclusion **19**

A Appendix A - Data Structures **20**

B Appendix B - Proofs **27**

C Appendix C - Formal Security Analysis **29**
C.1 Security Model . . . 29
C.2 Hardness Assumption . . . 30
C.3 Main Theorem . . . 31
C.4 Key Lemmas . . . 34
C.5 PEACE-Specific Differences from Wang–Cao . . . 36
C.6 Discussion and Limitations . . . 37

Bibliography **38**

1 Abstract

In this report, we introduce the PEACE protocol, an ECIES-based, multi-hop, unidirectional proxy re-encryption scheme for the Cardano blockchain. PEACE solves the encrypted-NFT problem by providing a decentralized, open-source protocol for transferable encryption rights, enabling creators, collectors, and developers to manage encrypted NFTs without relying on centralized decryption services. Building on the Wang–Cao PRE scheme, PEACE introduces SNARK-verified witness binding, token-name ciphertext non-malleability, and constant on-chain storage, and provides a formal IND-CCA security proof. This work fills a gap in secure, private access to NFTs on Cardano. Project Catalyst[1] funded the PEACE protocol in round 14.

2 Introduction

Current NFT standards on Cardano either leave data unencrypted, available to anyone who views the NFT, or require centralization, with a company handling encryption on behalf of users. Current solutions [1] claim to offer decentralized encrypted assets (DEAs), but lack a publicly available, verifiable cryptographic protocol or an open-source implementation. Most, if not all, of the mechanics behind current DEA solutions remain undisclosed. This report aims to fill that knowledge gap by providing an open-source implementation of a decentralized re-encryption protocol for encrypted assets on Cardano.

The protocol must allow tradability of both the NFT and the right to decrypt its data, which requires smart contracts paired with an encryption scheme that can re-encrypt for a new user without revealing the content. The tokens must be soulbound to ensure decryptability. Proxy re-encryption (PRE), the process of re-encrypting a ciphertext from one key to another without decrypting, has been studied extensively [2] [3] [4], and patented cloud-based implementations exist [5]. However, there are no open-source, fully on-chain, decentralized re-encryption protocols for encrypted NFT data on Cardano. The PEACE protocol fills this gap.

The PEACE protocol implements a unidirectional, multi-hop proxy re-encryption (PRE) scheme inspired by Wang and Cao [6] that is ECIES-based [7] and uses AES [8]. Unidirectionality means that Alice can re-encrypt for Bob, and Bob can then re-encrypt it back to Alice, using different encryption keys. Unidirectionality is important for tradability, as it defines the one-way flow of data and removes any restriction on who can purchase the NFT. Multi-hop means that the flow of encrypted data from Alice to Bob to Carol, and so on, does not end; the data can always be re-encrypted for a new user. Multi-hopping is important for tradability, as a finitely tradable asset does not fit many use cases. Typically, an asset should always be tradable if the user wants to trade it. The encryption primitives used in the protocol are considered industry standards at the time of this report.

While the re-encryption core builds on Wang and Cao's scheme, the original construction was presented as a poster without a full security proof, and subsequent work in the PRE literature has identified CCA vulnerabilities in schemes of this family when the algebraic bindings are insufficient. The PEACE protocol addresses these vulnerabilities through several extensions not present in the original construction: a Groth16 SNARK that binds the re-encryption witness to the pairing secret, preventing arbitrary witness substitution; token-name binding in the ciphertext verification equation, which ties each ciphertext to a specific on-chain NFT and prevents cross-token transplant attacks; binding Σ-protocols that enable CCA decryption simulation via witness extraction; constant

[1] https://projectcatalyst.io/funds/14/cardano-use-cases-concepts/decentralized-on-chain-data-encryption

on-chain datum size regardless of hop count, achieved by storing only the current and previous encryption levels; and a formal IND-CCA security proof (Appendix C) establishing that PEACE achieves CCA security under the PRE-DDH assumption in the generic group model.

The remainder of this report is as follows. Section 2 discusses the preliminaries and background required for this project. Section 3 provides a brief overview of the required cryptographic primitives and positions PEACE within the PRE literature. Section 4 gives a detailed description of the protocol. Section 5 examines security and threat analysis, the protocol's limitations, and related topics. The goal of this report is to serve as a complete reference and description of the PEACE protocol.

3 Background And Preliminaries

Understanding the protocol will require some technical knowledge of modern cryptographic methods, a basic understanding of elliptic curve arithmetic, and a general understanding of how smart contracts work on the Cardano blockchain. Anyone comfortable with these topics will find this report very useful and easy to follow. The report will attempt to use research standards for terminology and notation. The elliptic curve used in this protocol will be BLS12-381 [9]. Aiken is used to write all smart contracts required for the protocol [10].

Table 1: Symbol Description [11]

Symbol	Description
n	The order of $E(\mathbb{F}_p)$
r	A prime number dividing n
δ	A non-zero integer in $\mathbb{Z}_n$
$\mathbb{G}_1$	A subgroup of order r of $E(\mathbb{F}_p)$
$\mathbb{G}_2$	A subgroup of order r of the twist $E'(\mathbb{F}_{p^2})$
$\mathbb{G}_T$	The multiplicative target group of the pairing: $\mu_r \subset \mathbb{F}^*_{p^{12}}$
$e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$	A type-3 bilinear pairing
q	A fixed generator in $\mathbb{G}_1$
p	A fixed generator in $\mathbb{G}_2$
R	The Fiat–Shamir transformer
H	A hash function
m	The order of Ed25519
h_0, h_1, h_2, h_3	Fixed public points in $\mathbb{G}_2$ with unknown discrete logarithms
γ	A non-zero integer in $\mathbb{Z}_m$

Notation note. The symbol p in Table 1 denotes the fixed $\mathbb{G}_2$ generator throughout the protocol; the field characteristic of BLS12-381 is not referenced by symbol in any formula. Similarly, r in the symbol table denotes the prime subgroup order, while r appearing as a local variable in algorithms (e.g., a random nonce in Algorithms 1–5) is an independent value; context and scope distinguish the two uses.

The protocol, including both on-chain and off-chain components, will heavily use the `Register` type shown in Listing 1. The `Register` stores a generator, $g \in \mathbb{G}_\kappa$ and the corresponding public value $u = [\delta]g$ where $\delta \in \mathbb{Z}_n$ is a secret. We shall assume that the hardness of ECDLP and CDH in $\mathbb{G}_1$

will result in the inability to recover the secret δ. When using a pairing, we additionally rely on the standard bilinear Diffie–Hellman assumptions over the subgroups ($\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$). We will represent the groups $\mathbb{G}_1$ and $\mathbb{G}_2$ with additive notation and $\mathbb{G}_T$ with multiplicative notation.

Where required, we will verify Ed25519 signatures for cost-minimization, as relying solely on pure BLS12-381 for simple signatures becomes too costly on-chain. There will be instances where the Fiat–Shamir transform [12] will be applied to a Σ-protocol to transform it into a non-interactive variant. Concrete parameter choices are given in the Parameter Selection subsection below.

3.1 Parameter Selection

The PEACE protocol instantiates its cryptographic primitives with the following concrete parameters:

- **Elliptic curve:** BLS12-381 with a 255-bit scalar field of order $r \approx 2^{255}$
- **Signing:** Ed25519 with 32-byte secret keys [13]
- **General hashing:** Blake2b-224, producing 28-byte (224-bit) digests [14]
- **Key derivation:** HKDF with SHA3-256, producing 32-byte output keys
- **Symmetric encryption:** AES-256-GCM with 96-bit (12-byte) random nonces; the 256-bit key is the HKDF output
- **Arithmetic-circuit hash:** MiMC over the BLS12-381 scalar field $\mathbb{F}_r$, 91 rounds, with domain tag `F12|To|Hex|v1|`
- **Schnorr proof domain tag:** `SCHNORR|PROOF|v1|`
- **Binding proof domain tag:** `BINDING|PROOF|v1|`
- **SNARK validity window:** 1 hour; pending TTL: 6 hours
- **Fixed $\mathbb{G}_2$ points:** $h_0, h_1, h_2, h_3 \in \mathbb{G}_2$ with unknown discrete logarithms, generated deterministically via hash-to-curve
- **Groth16 circuit:** 1,084,616 constraints with 36 public inputs

4 Cryptographic Primitives Overview

This section provides brief explanations of the cryptographic primitives required by the protocol. If a primitive has an algorithmic description, then it will be included in the respective sub-section. We will represent the `Register` type as a tuple, (g, u), for simplicity inside the algorithms. We shall assume that the compression and uncompression of the elliptic curve points are canonical and follow the Zcash implementation [15]. Correctness proofs for many algorithms are in Appendix B.

4.1 Register-based

The protocol requires proving knowledge of a user's secret using a Schnorr Σ-protocol [16] [17]. This algorithm is both complete and zero-knowledge. We can use simple Ed25519 signatures for spendability, and then use the Schnorr Σ-protocol for knowledge proofs related to encryption. We will make the protocol non-interactive via the Fiat–Shamir transform, R, as shown in Listing 18.

Algorithm 1: Non-interactive Schnorr's Σ-protocol for the discrete logarithm relation

Input:
(g, u) where $g \in \mathbb{G}_\kappa$, $u = [\delta]g \in \mathbb{G}_\kappa$
Output: bool

select a random $r \in \mathbb{Z}_n$
compute $a = [r]g$
calculate $c = R(g, u, a)$
compute $z = \delta \cdot c + r$
output $[z]g = a + [c]u$

The protocol requires proving a binding relationship between a user's public value and other known elliptic-curve elements. The binding proof is a combination of multiple Schnorr's Σ-protocols. The value χ is the $\mathbb{G}_1$ element of the second term in the encryption level, specifically, `r2_g1b`.

Algorithm 2: Non-interactive Binding Σ-protocol

Input:
(g, u) where $g \in \mathbb{G}_1$, $u = [\delta]g \in \mathbb{G}_1$
(r_1, χ) where $r_1 \in \mathbb{G}_1$, $\chi \in \mathbb{G}_1$
(a, r) where $a \in \mathbb{Z}_n$ and $r \in \mathbb{Z}_n$
Output: bool

select a random $\rho \in \mathbb{Z}_n$ and $\alpha \in \mathbb{Z}_n$
compute $t_1 = [\rho]g$
compute $t_2 = [\alpha]g + [\rho]u$
calculate $c = R(g, u, t_1, t_2)$
compute $z_a = a \cdot c + \alpha$
compute $z_r = r \cdot c + \rho$
output $[z_a]g + [z_r]u = t_2 + [c]\chi \wedge [z_r]g = t_1 + [c]r_1$

There will be times when the protocol requires proving some equality using pairings. In these cases, we can use something akin to the BLS signature scheme, allowing only someone with the knowledge of the secret to prove the pairing equivalence. BLS signatures are a straightforward yet important signature scheme for the protocol, as they enable public confirmation of knowledge of a complex relationship beyond the limitations of Schnorr's Σ-protocols. BLS signatures work because of the bilinearity of the pairing [18].

Algorithm 3: Boneh-Lynn-Shacham (BLS) signature method

Input:
(g, u, c, w, m) where $g \in \mathbb{G}_1$, $u = [\delta]g \in \mathbb{G}_1$,
$c = p^{H(m)} \in \mathbb{G}_2$, $w = [\delta]c \in \mathbb{G}_2$, and $m \in \{0, 1\}^*$
Output: bool

$e(u, c) = e(g, w)$
$e(q^\delta, c) = e(q, c^\delta)$

4.2 ECIES + AES-GCM

The Elliptic Curve Integrated Encryption Scheme (ECIES) is a hybrid protocol involving asymmetric cryptography with symmetric ciphers. The encryption used in ECIES is the Advanced Encryption Standard (AES). ECIES and AES, combined with a key derivation function (KDF) such as HKDF

[19], form a complete encryption system. For readability, we present a simplified ECIES sketch here. The reference implementation provides the exact PEACE instantiation and serialization rules.

Algorithm 4: Encryption using ECIES + AES

Input:
(g, u) where $g \in \mathbb{G}_\kappa$, $u = [\delta]g \in \mathbb{G}_\kappa$, $m \in \{0,1\}^*$
Output: (r, c, h)

select a random $\rho \in \mathbb{Z}_n$
compute $r = [\rho]g$
compute $s = [\rho]u$
generate $k = KDF(s|r)$
encrypt $c = AES(m, k)$
compute $h = BLAKE2B(m)$
output (r, c, h)

Decrypting the ciphertext requires rebuilding the data encryption key (DEK), k, from the KDF. The DEK is rebuildable because r is public and the user knows the secret δ, allowing them to decrypt the data.

Algorithm 5: Decryption using ECIES + AES

Input:
(g, u) where $g \in \mathbb{G}_1$, $u = [\delta]g \in \mathbb{G}_1$,
(r, c, h) as the capsule
Output: ($\{0,1\}^*$,bool)

compute $s' = [\delta]r$
generate $k' = KDF(s'|r)$
compute $m' = AES(c, k')$
compute $h' = BLAKE2B(m')$
output (m', h' = h)

Algorithm 4 describes the case where a `Register` is used to generate the DEK from the KDF function. Anyone with knowledge of k may decrypt the ciphertext. The algorithm shown differs slightly from the PEACE protocol's implementation, as the protocol allows transferring the DEK to another `Register`; however, the general flow remains the same. The key takeaway here is that encrypting a message and decrypting the resulting ciphertext require a key to generate the DEK. Both algorithms 4 and 5 use a simple hash function for authentication. In the PEACE protocol, we will use AES-GCM with authenticated encryption with associated data (AEAD) for authentication.

4.3 Re-Encryption

There are various types of re-encryption schemes, ranging from classical proxy re-encryption (PRE) to hybrid methods. These re-encryption schemes involve a proxy, an entity that performs both re-encryption and verification. The PRE used in the PEACE protocol is modeled as an interactive flow between the current owner and a prospective buyer, using a smart contract as part of the proxy. We need an interactive scheme because, in many real-world use cases, numerous off-chain checks, such as KYC/AML regulations and various legal requirements, must occur before transferring the right to decrypt on-chain to the new owner. The PEACE protocol obtains interactivity via a bidding system that requires the current owner to agree to the exchange.

The method described below is a hybrid approach. The current owner's wallet performs the re-encryption process for the buyer. At the same time, the Cardano smart contract acts as a proxy, verifying various cryptographic proofs, enforcing the correct bindings, handling payments, and updating the on-chain owner fields. This design explicitly supports off-chain processes before the transfer of decryption rights. The current owner only submits the re-encryption transaction once these off-chain conditions are satisfied. This method will support the broadest range of real-world asset use cases. The PRE is unidirectional, meaning the re-encryption flow is one-way: from the current owner to the next owner. If Alice delegates to Bob, Bob does not automatically gain the ability to 'go backwards' and create ciphertexts for Alice using the same re-encryption material. This flow differs from a bidirectional method, where the PRE is symmetric, enabling a two-way encryption relationship between the parties, meaning Alice can transform a ciphertext into one for Bob, and Bob can transform a ciphertext into one for Alice, without either Alice or Bob having to re-run the entire re-encryption flow. That is not what we want for this implementation. Each direction is a separate, explicit transfer of rights with its own re-encryption material, meeting the protocol's tradability requirements.

Note that in the original Catalyst proposal, the protocol defines itself as a bidirectional, multi-hop PRE. However, during the design phase, it became clear that the actual Cardano use case requires a unidirectional, multi-hop PRE. This change is fully compatible with the original proposal's PRE goals (transfer of decryption rights without exposing plaintext or private keys), but reflects the reality of trading tokens via Cardano smart contracts within the PRE landscape.

The algebraic intuition behind Algorithm 6 is as follows. The fresh random scalar a seeds the pairing secret $\kappa = e(q^a, h_0) \in \mathbb{G}_T$, which becomes the next hop's decryption key material after hashing. The second random scalar r re-randomizes Bob's half-level: $r_{1,b} = [r]q$ is a fresh base point, and $r_{2,b} = [a]q + [r]v$ hides a under Bob's public key $v = [\delta_b]q$, so only Bob can recover a (and hence κ) using his secret δ_b. The chain-link term $R_5 = [H(\kappa)]p + [\delta_a](-h_0)$ encodes $H(\kappa)$ in $\mathbb{G}_2$, offset by Alice's secret; only someone knowing δ_a can recover $H(\kappa)$ from R_5 via the pairing equation $e(q, R_5) \cdot e(u_a, h_0) = e(W, p)$. The SNARK (Algorithm 7) separately proves that $W = [H(\kappa)]q$ was honestly derived from κ, preventing arbitrary witness substitution.

Algorithm 6: Owner-mediated re-encryption from Alice to Bob

Input:
(q, u) where $q \in \mathbb{G}_1$, $u = [\delta_a]q \in \mathbb{G}_1$ (Alice's public key),
(q, v) where $v = [\delta_b]q \in \mathbb{G}_1$ (Bob's public key),
Alice's secret key $\delta_a \in \mathbb{Z}_n$
$p \in \mathbb{G}_2$
Alice's current `HalfEncryptionLevel`: $(r_{1,a}, r_{2,a}, r_{4,a})$, where $r_{1,a}, r_{2,a} \in \mathbb{G}_1$ and $r_{4,a} \in \mathbb{G}_2$
(h_0, h_1, h_2, h_3), where $h_i \in \mathbb{G}_2$ are fixed public points with unknown discrete logarithms.
Output: Bob's `HalfEncryptionLevel`: $(r_{1,b}, r_{2,b}, r_{4,b})$
Alice's `FullEncryptionLevel`: $(r_{1,a}, r_{2,a}, r_5, r_{4,a})$

select a random $a \in \mathbb{Z}_n$
compute $\kappa = e(q^a, h_0)$
select a random $r \in \mathbb{Z}_n$
compute $r_{1,b} = [r]q$
compute $r_{2,b} = [a]q + [r]v \in \mathbb{G}_1$
compute $c = [BLAKE2b(r_{1,b})]h_1 + [BLAKE2b(r_{1,b}||r_{2,b}||\mathsf{token})]h_2$
compute $r_{4,b} = [r]c$
compute $r_5 = [H(\kappa)]p + [\delta_a](-h_0) \in \mathbb{G}_2$
output $(r_{1,b}, r_{2,b}, r_{4,b})$ and $(r_{1,a}, r_{2,a}, r_5, r_{4,a})$

Note on h_3 omission. At level $k > 1$ (re-encryption), the R_4 verification uses $c = [H(R_1)]h_1 + [H(R_1\|R_2\|\mathsf{token})]h_2$ without the h_3 term. At level~1 (initial encryption, Listing~7), the equation includes h_3: $c = [H(R_1)]h_1 + [H(R_1\|R_2\|\mathsf{token})]h_2 + h_3$. This intentional difference provides domain separation between first-level and re-encrypted ciphertexts; see the Game~1 verification equation in Appendix~C where the level distinction is explicit.

Algorithm 6 describes the actual re-encryption process for Alice, resulting in the transfer of the decryption rights to Bob. Bob can then use this information to recursively calculate the secret κ and eventually the original secret used in the encryption process.

4.4 Groth16 SNARK Verification

The protocol uses a Groth16 zero-knowledge proof to ensure that the re-encryption witness $W = q^{H(\kappa)}$ is correctly derived from the secret $\kappa = e(q^a, h_0)$. The SNARK circuit proves the following statement without revealing the secrets a or r:

Algorithm 7: SNARK Circuit Statement

Input:
Secret inputs: (a, r) where $a, r \in \mathbb{Z}_n$
Public inputs: (v, w_0, w_1) where $v, w_0, w_1 \in \mathbb{G}_1$
Output: bool

compute $\kappa = e([a]q, h_0)$
compute $hk = MiMC(\kappa\|DomainTag)$
compute $p_0 = [hk]q$
compute $p_1 = [a]q + [r]v$
output $w_0 = p_0 \wedge w_1 = p_1$

The public inputs v, w_0, and w_1 are provided as affine $\mathbb{G}_1$ coordinates. The on-chain verifier receives

these coordinates as 64-bit limbs and reconstructs the compressed $\mathbb{G}_1$ points via limb compression verification. This binding ensures that the SNARK public inputs correspond to the actual elliptic curve points stored in the datum.

The Groth16 verifier uses the gnark commitment extension, which adds a Pedersen commitment proof-of-knowledge check alongside the standard pairing verification. The verification equation is:

$$e(A, B) \cdot e(vk_x, -\gamma) \cdot e(C, -\delta) \cdot e(\alpha, -\beta) = 1$$

where vk_x incorporates both the public inputs and commitment wires. (Here $\alpha, \beta, \gamma, \delta$ are Groth16 CRS elements, distinct from the user secret δ defined in Table~1.)

4.5 Related Work And Design Rationale

Proxy re-encryption was introduced by Mambo and Okamoto [2] and formalized by Blaze, Bleumer, and Strauss [3], who proposed the first bidirectional, single-hop PRE scheme. Ateniese et al. [4] [20] improved upon this with unidirectional, single-hop constructions based on bilinear pairings. Canetti and Hohenberger [21] gave the first CCA-secure PRE construction, proving security in the standard model under the decisional bilinear Diffie–Hellman (DBDH) assumption; however, their scheme is bidirectional and single-hop, which does not meet the multi-hop tradability requirement. Libert and Vergnaud [22] achieved CCA-secure unidirectional PRE under the decision linear (DLIN) assumption, but their construction is also single-hop. Neither Canetti–Hohenberger nor Libert–Vergnaud support multi-hop re-encryption, which is essential for an NFT that may be traded an unbounded number of times.

Wang and Cao [6] proposed a "fully secure unidirectional and multi-use" PRE scheme at a CCS 2009 poster session. Their construction is the closest prior work to PEACE: it is unidirectional, multi-hop, and pairing-based. However, the Wang–Cao construction was published as a poster without a full security proof, and the PRE literature has since identified CCA vulnerabilities in schemes of this family when the algebraic bindings between ciphertext components are insufficient to prevent malleability. PEACE builds on the Wang–Cao re-encryption core but addresses these vulnerabilities through SNARK witness binding, token-name ciphertext non-malleability, and binding Σ-protocols, achieving formal IND-CCA security (Appendix C).

IronCore Labs' Recrypt library [5] is an open-source, Rust-based PRE implementation used in cloud-based encryption products. Recrypt implements a transform-based PRE where a semi-trusted proxy applies re-encryption keys. While production-quality software, Recrypt's model assumes a centralized proxy service and does not target the on-chain verification constraints of a blockchain. PEACE differs architecturally: re-encryption is owner-mediated (Section~4.1), and all cryptographic proofs are validated on-chain by passive validators, removing the trusted-proxy assumption.

No prior work provides an open-source, fully on-chain, multi-hop, unidirectional PRE with formal CCA security guarantees on a UTxO-based blockchain. Table 2 summarizes the landscape.

Table 2: Comparison of PRE schemes relevant to PEACE

Scheme	Direction	Hops	CCA	On-chain	Open source
Blaze et al. '98	Bi	1	No	No	—
Ateniese et al. '05	Uni	1	No	No	—
Canetti–Hohenberger '07	Bi	1	Yes	No	—
Libert–Vergnaud '08	Uni	1	Yes	No	—
Wang–Cao '09	Uni	Multi	No*	No	—
IronCore Recrypt	Uni	Multi	—	No	Yes
PEACE	**Uni**	**Multi**	**Yes**	**Yes**	**Yes**

*Claimed but no full proof published; CCA vulnerabilities identified in this family.

5 Protocol Overview

The PEACE protocol is an ECIES-based, multi-hop, unidirectional proxy re-encryption scheme for the Cardano blockchain, allowing creators, collectors, and developers to trade encrypted NFTs without relying on centralized decryption services. The current implementation uses the Cardano blockchain as the data storage layer. Cardano's chain parameters limit on-chain storage to less than 16 KB. For use cases requiring larger payloads, the data storage layer can be extended to support arbitrary file sizes.

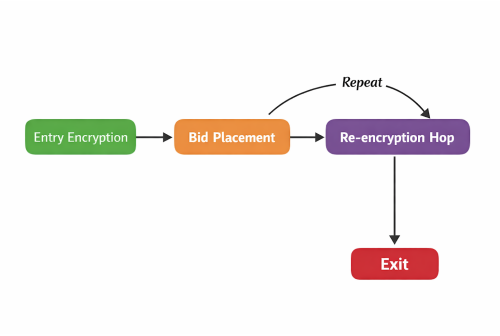

Figure 1: Protocol state flow

5.1 Design Goals And Requirements

Two equally important areas, the on-chain and off-chain, define the protocol design. The on-chain design is everything related to smart contracts written in Aiken for the Cardano blockchain. The off-chain design includes transaction construction, cryptographic proof generation, and the happy-path flow. The design on both sides will focus on a two-party example: Alice and Bob, who want to trade encrypted data. Alice will be the original owner, and Bob will be the new owner. The current off-chain implementation focuses on the two-party case. Extending to a general n-party system is planned as future work.

The protocol must allow continuous trading via a multi-hop PRE, meaning that Alice trades with Bob, who can then trade with Carol. In this setting, Alice will trade to Bob, then Bob will trade back to Alice, rather than Carol, without any loss of generality. Each hop will generate a new owner and decryption data for the encryption UTxO. The on-chain storage remains constant per hop: the datum stores only the current half-level and the previous full-level, rather than a growing list of all levels. Users reconstruct the complete encryption level history by querying the blockchain from the

mint transaction to the current state. Users will use a bid system for token trading rather than a fixed-price listing model. This is a deliberate design choice driven by the nature of encrypted content as an opaque good: the buyer cannot inspect the content before purchase, so the buyer's willingness to pay reflects their trust in the seller and the asset's provenance rather than direct content evaluation. A bid system allows the market to price opaque goods through negotiation. The buyer names a price they consider fair given the available signals, such as seller reputation, asset metadata, and historical trade count, and the seller accepts or ignores the bid. This flexibility also allows sellers to accept below-asking bids for assets that have been listed for extended periods, accommodating natural price discovery over time. Content quality assurance is orthogonal to the re-encryption protocol and is better addressed by external reputation systems. A user may choose not to trade their token by simply not selecting a bid if one exists.

The protocol implements a state machine with two states: `Open` and `Pending`. In the `Open` state, the owner may remove the encryption UTxO, submit a SNARK proof to transition to `Pending`, or simply hold the asset. In the `Pending` state, a bidder's re-encryption request is awaiting completion; the owner may complete the re-encryption (transitioning back to `Open` with a new owner), cancel the pending request, or allow the request to expire after a time-to-live (TTL) threshold. This state machine ensures atomicity of the SNARK verification and re-encryption steps while allowing recovery from abandoned transactions.

The re-encryption process is unidirectional (Section~1): each hop is a one-way transfer of decryption rights, preventing the buyer from reversing the flow without initiating a new trade. Any bidirectionality would imply symmetry between Alice and Bob, circumventing the re-encryption requirement. The unidirectional requirement forces tradability to follow the typical trading interactions currently found on the Cardano blockchain.

Each UTxO in this system must be uniquely identified via an NFT. The uniqueness requirement works well for the encryption side because the NFT could be a tokenized representation of the encrypted data, something similar to a CIP68 [23] contract, but using a single token. The bid side does work, but the token becomes a pointer rather than having any real data attached, essentially a unique, one-time-use token. Together, they provide the correct uniqueness requirement. UTxOs may be removed from the contract at any time by the owner. After the trade, the owner of the encrypted data may do whatever they want with that data. The protocol does not require the re-encryption contract to permanently store the encrypted data.

The protocol will use an owner-mediated re-encryption flow (a hybrid PRE), which is UX-equivalent to a classical proxy re-encryption scheme in this setting, since smart contracts on Cardano are passive validators and do not initiate actions. Ultimately, some user must act as the proxy, the one doing the re-encryption, because the contract cannot do it on its own. The smart contract must act as the proxy's validator, not solely as the proxy itself. In the current implementation, the owner acts as their own proxy in the protocol.

5.2 On-Chain And Off-Chain Architecture

There will be two user-focused smart contracts: one for re-encryption and the other for bid management. Any UTxO inside the re-encryption contract is for sale via the bidding system. A user may place a bid into the bid contract, and the current owner of the encrypted data may select it as payment for re-encrypting the data to the new owner. To ensure functionality, a reference data contract must exist, as it resolves circular dependencies. The reference datum will contain the script hashes for the re-encryption and bid contracts. Data structures and functions are in Appendix A.

The bid contract datum structure is shown in Listing 2. The bid datum contains all of the required information for re-encryption. The owner of a bid UTxO will be type `Register` in $\mathbb{G}_1$. The `pointer` is the NFT name on the bid UTxO, and `token` is the NFT name on the re-encryption UTxO. The `token` forces the bid to only apply to a specific sale.

The bid contract redeemer structures are shown in Listing 3. Entering into the bid contract uses the `EntryBidMint` redeemer, triggering a `pointer` mint validation, a `token` UTxO existence check, an Ed25519 signature with `owner_vkh`, and a Schnorr Σ-protocol using `owner_g1`. Leaving the bid contract requires using `RemoveBid` and `LeaveBidBurn` redeemers together, triggering a `pointer` burn validation and Ed25519 signature with `owner_vkh`. When a user selects a bid, they will use `UseBid` and `LeaveBidBurn` together, triggering a `pointer` burn validation and the proxy re-encryption validation.

Listing 4 shows the re-encryption contract datum structure. The re-encryption datum contains all of the required information for decryption. The owner of the re-encryption UTxO will be type `Register` in $\mathbb{G}_1$. The `token` is the NFT name on the re-encryption UTxO. The ciphertext and related data are in the `Capsule` subtype. Rather than storing all encryption levels, the datum stores only the current `half_level` and optionally the previous `full_level`. Users reconstruct the complete level history by querying the blockchain. The `status` field tracks the state machine position: `Open` for normal operation, or `Pending` with the SNARK public inputs and TTL when awaiting re-encryption completion.

The re-encryption contract redeemer structures are shown in Listing 5. Entering into the re-encryption contract uses the `EntryEncryptionMint` redeemer, triggering a `token` mint validation, an Ed25519 signature with `owner_vkh`, a binding proof using `owner_g1`, and a Schnorr Σ-protocol using `owner_g1`. Leaving the re-encryption contract requires using both `RemoveEncryption` and `LeaveEncryptionBurn` redeemers, triggering a `token` burn validation and an Ed25519 signature with `owner_vkh`. The `UseSnark` redeemer initiates the re-encryption process by verifying the Groth16 proof and transitioning the status from `Open` to `Pending`. The `UseEncryption` redeemer completes the re-encryption by verifying limb compression, updating ownership, and transitioning back to `Open`. The `CancelEncryption` redeemer allows the owner to cancel a pending re-encryption or allows automatic cancellation after TTL expiration.

The re-encryption flow requires two transactions: first `UseSnark` to submit the SNARK proof, then `UseEncryption`, `UseBid`, and `LeaveBidBurn` together to complete the transfer.

5.3 Key Management And Identity

Each user in the protocol can deterministically generate BLS12-381 key pairs represented by a `Register` value in $\mathbb{G}_1$. The $\mathbb{G}_1$ points are the user's on-chain identity for encryption and signature verification. The corresponding secret scalar, $\delta \in \mathbb{Z}_n$, is held off-chain by the user's wallet or client software and never published on-chain.

The BLS12-381 keys used for re-encryption are logically separate from the Ed25519 keys used to sign Cardano transactions. A wallet must manage both Ed25519 keys for authorizing UTxO spending and BLS12-381 scalars for obtaining and delegating decryption rights. Losing or compromising the BLS12-381 secret key means losing the ability to decrypt any items associated with that identity, even if the Cardano spending keys are still available.

Key rotation is structurally constrained in a multi-hop PRE chain. Each encryption level embeds the owner's $\mathbb{G}_1$ public value at the time of that hop. Rotating the BLS12-381 key would require

re-encrypting every historical level in the chain, which defeats the purpose of the re-encryption scheme and is not feasible on a public ledger where historical levels are immutable. If a user's BLS12-381 secret key is compromised, an attacker can decrypt all current and future capsules addressed to that key, but cannot retroactively remove or alter on-chain history. The owner can remove their encryption UTxOs from the contract (burning the tokens), but cannot retroactively protect data that was encrypted under the compromised key. The practical mitigation is key hygiene: users should protect their BLS12-381 secret scalar with the same rigor applied to any long-term cryptographic key. Revocation and migration mechanisms remain an open problem for multi-hop PRE schemes generally, not specific to PEACE.

For each encrypted item, the protocol generates a fresh KEM used at each encryption level. The KEM is never directly stored on-chain. The on-chain Capsule contains the AES-GCM nonce, associated data, and ciphertext.

5.4 Protocol Specification

5.4.1 Phase 1: Creating The Encryption UTxO

The protocol flow starts with Alice selecting a secret $[\gamma] \in \mathbb{Z}_m$ and $[\delta] \in \mathbb{Z}_n$. The secret γ will generate an Ed25519 keypair. The secret δ will generate the `Register` in $\mathbb{G}_1$ using the fixed generator, q. Alice will fund the address associated with the `VerificationKeyHash`, the `vkh`, of the Ed25519 keypair with enough Lovelace to pay for the minimum required Lovelace for the contract UTxO, the change UTxO, and the transaction fee. Alice may then build the re-encryption entry transaction.

The re-encryption entry transaction will contain a single input and two outputs. The transaction will mint a `token` using the `EntryEncryptionMint` redeemer. The `token` name is generated by the concatenation of the input's output index and transaction ID, as shown in Listing 6. The protocol specification assumes a single input, but this transaction may use multiple inputs. If more than one input exists, then the first element of a lexicographically sorted input list will be used for the name generation.

Alice may now finish building the `EncryptionDatum` by constructing the `half_level` and `capsule` fields. Since Alice is the first owner, she will encrypt it for herself. Alice will encrypt the original data by generating a root secret $\kappa_0 \in \mathbb{G}_T$. The root secret, κ_0, will be used in the KDF to produce a valid DEK. The message will be encrypted using AES-GCM. The `Capsule` type will store the resulting information. Listing 7 is a Pythonic pseudocode for generating the original encrypted data and the first encryption level. The sub-types of the `EncryptionDatum` can be populated as shown in Listing 8. The contract will validate the first half-level using the assertion from Listing 9. Alice can prove to herself that the encryption level is valid by verifying the assertion in Listing 10. Alice may now construct the full `EncryptionDatum` as shown in Listing 11, with `status` set to `Open` and `full_level` set to `None`.

The entry redeemer verifies that Alice's `owner_vkh` is valid via an Ed25519 signature, since Alice needs a valid `vkh` to remove the entry. The entry redeemer also verifies a valid `Register` via a Schnorr Σ-protocol as shown in Algorithm 1. Alice needs this to decrypt her own data and to verify that she binds her public value to the first encryption level via a binding proof, as shown in Algorithm 2. The encrypted data is ready to be traded after successfully creating a valid entry transaction and submitting it to the Cardano blockchain.

While the encryption UTxO's `status` remains `Open`, Alice may update her asking price using the

`UpdateEncryptionPrice` spend redeemer. This requires Alice's signature and enforces that the new price is non-negative. Only the `new_price` field may change; all other datum fields and non-ADA tokens on the UTxO must remain constant. This allows Alice to adjust her asking price without recreating the encryption UTxO.

5.4.2 Phase 2: Creating The Bid UTxO

Bob may now create a bid UTxO in the bid contract to purchase the encrypted data from Alice. First, Bob selects a secret $[\gamma] \in \mathbb{Z}_m$ and $[\delta] \in \mathbb{Z}_n$. Similarly to Alice, the secret γ will generate an Ed25519 keypair and the secret δ will generate the `Register` in $\mathbb{G}_1$ using the fixed generator, q. Bob will fund the address associated with his `vkh` with enough Lovelace to pay for payment, the change UTxO, and the transaction fee. Bob may then build the bid entry transaction. Note that the on-chain datum size remains constant regardless of hop count, as only the current half-level and previous full-level are stored. Bob should contribute to the minimum required Lovelace for the encrypted data, though this is not an on-chain requirement.

The structure of the bid entry transaction is similar to that of the re-encryption entry transaction, but uses `EntryBidMint` instead of `EntryEncryptionMint`. The transaction input derives the `pointer` token name in the same way as the `token` name. A user may reference the `token` name on-chain from the re-encryption contract. Bob may then create the `BidDatum` as shown in Listing 12.

Similar to the re-encryption contract, the entry redeemer will verify Bob's `vkh` and the `Register` values in $\mathbb{G}_1$. A valid `Register` is important, as the validity of the $\mathbb{G}_1$ point determines whether Bob can decrypt the data after the re-encryption process. The `new_price` field in the `BidDatum` represents the price Bob is willing to pay for Alice to re-encrypt the data to his `Register`. There may be many bids, but Alice may only select a single bid for the re-encryption transaction. Bob may update the price of an existing bid using the `UpdateBidPrice` spend redeemer, which requires Bob's signature and enforces that the new price is non-negative. Only the `new_price` field may change; all other datum fields and non-ADA tokens on the UTxO must remain constant.

To mitigate grief attacks during the re-encryption flow, each bid is automatically locked for a minimum period (`minimum_bid_lock`, currently 6 hours) upon creation. The on-chain `EntryBidMint` redeemer enforces that `locked_until` is at least `minimum_bid_lock` in the future. The `RemoveBid` spend redeemer enforces that the transaction's validity interval lower bound exceeds `locked_until`. This guarantees Alice a safe window to compute the SNARK proof and submit both re-encryption transactions without Bob withdrawing his bid mid-process.

5.4.3 Phase 3: SNARK Submission And Re-Encryption

The re-encryption process occurs in two transactions to accommodate the computational cost of SNARK verification.

Transaction 1: SNARK Submission. Alice generates a Groth16 proof demonstrating that the witness $W = q^{H(\kappa)}$ is correctly derived from the pairing secret $\kappa = e(q^a, h_0)$. Alice submits this proof using the `UseSnark` redeemer, which verifies the Groth16 proof (via the groth_witness withdraw validator) and the commitment proof-of-knowledge. Upon successful verification, the datum's `status` transitions from `Open` to `Pending(groth_proof, groth_public, ttl)`, where `groth_proof` is the Groth16 proof, `groth_public` contains the SNARK public inputs (the limb representation of the G1 points), and `ttl` is the expiration time.

Transaction 2: Re-Encryption Completion. Alice selects a bid UTxO from the bid con-

tract and completes the re-encryption using the `UseEncryption` redeemer together with `UseBid` and `LeaveBidBurn`. This step burns Bob's bid token, updates the on-chain data to Bob's ownership, and creates the re-encryption proofs. The contract verifies limb compression to ensure the SNARK public inputs match the actual G1 points (`bid_owner_g1.public_value`, the witness, and `next_half_level.r2_g1b`). The PRE proofs demonstrate that the values produced by the re-encryption process match the expected values via pairings involving the original owner's `Register`, the new owner's `Register`, and the next encryption level. Additionally, the bid's `new_price` is transferred to the encryption datum's `new_price` field, recording the agreed-upon price on the encryption UTxO. If everything is consistent, then ownership and decryption rights transfer to Bob, and the status returns to `Open`.

Listing 13 is a Pythonic pseudocode for generating the next encryption level. Bob's half-level and Alice's full-level are shown in Listing 14. The complete next encryption datum is shown in Listing 15.

The contract will validate the re-encryption using a binding proof and two pairing proofs as shown in Listing 16. The first pairing assertion follows Alice's first-level validation, ensuring that the encryption level terms are consistent. The second pairing assertion shows that Alice correctly created the r_5 term. The SNARK proves that the secret κ truly does equal the witness when hashed, $W = q^{H(\kappa)}$, where $H(\kappa)$ and κ are secrets and W is public.

5.4.4 Phase 4: Decryption

Bob can now decrypt the root key by recursively computing all the random $\mathbb{G}_T$ points as shown in Listing 17. Since the on-chain datum stores only the current half-level and previous full-level, Bob must first reconstruct the complete encryption level history by querying the blockchain for all transactions from the token's mint to the current state. Each historical transaction contains the half-level and full-level at that point in time, allowing Bob to rebuild the complete list of encryption levels needed for recursive decryption.

6 Security Model

The PEACE protocol achieves formal IND-CCA security (Appendix C). This section describes the assumptions, trust model, threat analysis, and limitations that define the protocol's security boundary.

6.1 Assumptions

This protocol inherits standard assumptions from public-key cryptography and public blockchains. The assumptions below describe what must hold for the security claims in this document to be meaningful.

- All compressed curve points accepted by the protocol (on-chain or off-chain) MUST be validated as canonical encodings and as members of the correct prime-order subgroup, rejecting non-canonical encodings, the point at infinity where disallowed, and any point not in the intended subgroup. Otherwise an attacker may exploit small-subgroup/cofactor edge cases to bypass security claims or forge relations that appear to verify.
- Cryptographic assumptions hold: The security of the construction relies on hardness assumptions for the chosen primitives: the PRE-DDH assumption (Definition~C.2 in Appendix C),

which holds in the generic group model and is implied by CDH in $\mathbb{G}_1$; collision resistance and preimage resistance of the hash functions used (including domain separation); MiMC modeled as a random oracle within the SNARK circuit; and unforgeability of any signature schemes used.

- Correct domain separation: All hashes used for hashing-to-scalar, Fiat–Shamir transcripts, and key derivations use fixed domain tags and unambiguous encodings. A domain-separation bug is a critical security failure.
- Well-formed randomness: All secret scalars and nonces are sampled with high entropy and never reused where uniqueness is required. Randomness failures (poor RNG, nonce reuse, low-entropy secrets) are catastrophic.
- Endpoint key safety: Alice's and Bob's long-term secret keys must remain confidential. Extracting keys from the wallet/device undermines the confidentiality and authenticity guarantees.
- On-chain validation is authoritative: The ledger enforces the validator exactly as written (Aiken/Plutus semantics). Any check performed only off-chain is advisory and not part of the security boundary.
- Proof system assumptions: In a production setting, the protocol uses SNARKs. Their required assumptions hold (soundness and any additional properties needed for adversarial settings). If the SNARK requires a trusted setup, then the corresponding trapdoor ("toxic waste") is assumed destroyed.
- Chain security: The blockchain provides finality and censorship-resistance to the degree commonly assumed for Cardano. Prolonged reorgs, validator bugs, or sustained censorship are out of scope.
- Scope boundary: The protocol does not assume (and does not attempt to enforce) that the plaintext has any particular meaning or quality, nor does it assume Bob will keep plaintext private after decryption.

6.2 Trust Model

The protocol design minimizes trust between Alice and Bob. The smart contract is the source of truth for what constitutes a valid re-encryption hop. Anything not enforced by the on-chain validator is advisory. We do not assume Alice or Bob is honest. Either party may attempt to cheat, submit malformed data, or abort mid-protocol. Because PEACE uses owner-mediated re-encryption, there is no separate semi-trusted proxy: the owner acts as their own proxy, and the smart contract validates the re-encryption rather than performing it. Any compromised keys are a severe failure. We assume the hardness of the PRE-DDH assumption (Appendix C, Definition~C.2), collision resistance and preimage resistance of the hash functions, knowledge soundness of Groth16, and unforgeability of the signature schemes used. We assume the wallet or OS will protect endpoint key material. A compromise of long-term keys (Alice/Bob/etc) is out of scope except where explicitly mitigated (e.g., domain separation and on-chain binding checks).

6.3 Threat Analysis

Adversary capabilities

- Full network observer: can read all on-chain data, replay transactions, and correlate timing/amount patterns.

- Active attacker: can submit arbitrary transactions, craft malformed ciphertext/proofs, and attempt to use validator failures/success as an oracle.
- Insider attacker: Alice or Bob may act maliciously (sell garbage, withhold finalization, attempt to claim funds without delivering the correct re-encryption).
- Key compromise: theft of a participant's secret keys is possible.

Primary threats and production mitigations

- Invalid re-encryption accepted on-chain: mitigated by strict on-chain validation that binds ciphertext components, public keys, and transcript hashes to the expected relations.
- Related-ciphertext / CCA-style probing: mitigated by ciphertext non-malleability via the R_4 pairing binding (which ties ciphertext components together through hash-derived exponents), the binding Σ-protocol (which proves knowledge of the ciphertext's algebraic structure), and the SNARK witness binding (which prevents witness substitution). The combination of these mechanisms yields formal IND-CCA security (Appendix C).
- Multi-hop bypass: if downstream decryption reveals intermediate artifacts used to decrypt upstream ciphertexts without the proxy, it breaks the intended delegation boundary; mitigation is hop-level re-randomization / re-encapsulation and careful design to ensure decryption yields only plaintext (not reusable upstream capabilities).
- Fairness failure (abort/grief): either party can stop cooperating; mitigations are economic and protocol flow design, not cryptography alone.

6.4 Metadata Leakage

The protocol runs on a public UTxO ledger, so metadata leakage is unavoidable.

Potential leakage includes:

- Transaction graph linkage: repeated verification keys, registers, UTxO patterns, and timing can link multiple protocol runs to the same actors or workflow.
- Protocol fingerprinting: ciphertext sizes, hop count, and datum/redeemer structure can reveal which step the protocol is in and correlate participants across transactions.
- Value and timing leakage: amounts, fees, and time between steps can reveal trade size, urgency, and repeated counterparties.
- Key/identity linkage via commitments: even with hashed values, fixed-format commitments and domain tags can still be fingerprinted if reused or if inputs have low entropy.

Mitigations are partial and operational:

- Rotate addresses and avoid stable identifiers where possible
- Treat privacy as a separate layer (mixing, batching, relayers) rather than something the core protocol guarantees.

6.5 Limitations And Risks

- The protocol includes a Groth16 SNARK that proves the published $H(\kappa)$-dependent terms are derived from the actual pairing secret $\kappa = e(q^{a_0}, h_0)$. This SNARK enforces $hk = MiMC(e(q^{a_0}, h_0) \| DomainTag)$ and $W = q^{hk}$ without revealing κ, hk, or a_0. The SNARK

uses MiMC for hashing within the circuit, as it is efficient in arithmetic circuits. The limb compression verification binds the SNARK public inputs to the actual on-chain G1 points.

- The protocol can prove key-binding and correct re-encryption relations, but it cannot prove that the encrypted content is "valuable" or matches an off-chain description. Disputes about semantics require external mechanisms.
- The protocol does not protect the data after decryption. If Bob decrypts the plaintext, Bob can copy or leak it. Cryptography cannot prevent exfiltration. Only economic or legal controls can reduce this risk.
- The protocol does not ensure a perfectly fair exchange. Either party can abort at different stages. The two-transaction re-encryption flow introduces a window where Alice has submitted the SNARK proof but not yet completed the transfer. The TTL-based cancellation mechanism provides recovery but does not guarantee atomicity. To mitigate grief attacks where a buyer removes a bid during Alice's SNARK computation, bids are automatically locked for a minimum period (`minimum_bid_lock`) upon creation. This lock prevents the `RemoveBid` redeemer from succeeding until the lock expires, giving Alice a guaranteed window to compute and submit her proof. Combined with transaction chaining (submitting both re-encryption transactions back-to-back after offline proof generation), this eliminates the primary grief vector.
- Key compromise is catastrophic. Any theft of secret keys compromises the confidentiality of those assets. Losing secret keys prevents decrypting.
- The protocol achieves IND-CCA security through the combination of binding proofs, pairing checks, and SNARK verification (Appendix C). The security model is IND-CCA (not IND-PRE-CCA) because PEACE uses owner-mediated re-encryption (Section~4.1), so no standalone re-encryption key is published. Extending to full IND-PRE-CCA would require simulating re-encryption for the challenge user, which is infeasible in a Type-3 pairing without a homomorphism from $\mathbb{G}_1$ to $\mathbb{G}_2$; this remains an open problem for Wang–Cao style PRE on asymmetric pairings. The proof relies on the PRE-DDH assumption, a custom assumption justified in the generic group model and implied by CDH in $\mathbb{G}_1$ (Definition~C.2). MiMC is modeled as a random oracle within the SNARK circuit; replacing MiMC with Poseidon would not affect the proof structure.
- The protocol does not limit hops directly. However, since the on-chain storage is constant per hop (Section~4.1), the Cardano UTxO size limit does not constrain the hop count. Users must query the blockchain to reconstruct the full encryption history for decryption.
- Pairing-heavy verification and SNARK verification can approach the CPU budget, requiring multi-transaction validation flows, increasing complexity, and reducing UX reliability under network congestion.
- BLS12-381 and Ed25519 are not post-quantum secure. A sufficiently powerful quantum computer running Shor's algorithm could recover secret keys from public keys, breaking both the pairing-based re-encryption and the signature scheme. Because encrypted data is recorded on a public ledger, the protocol is vulnerable to "harvest now, decrypt later" attacks: an adversary could record on-chain ciphertexts today and decrypt them once large-scale quantum hardware becomes feasible. For assets intended to retain confidentiality over long time horizons, this is a material risk. Migration to post-quantum primitives (e.g., lattice-based PRE and hash-based signatures) would require changes to both the pairing-based re-encryption core and the SNARK system, and is left as future work.

6.6 Performance And On-Chain Cost

The re-encryption process performs well within Cardano's transaction limits by splitting the validation across two transactions. The generation of the SNARK proofs occurs off-chain and takes approximately three minutes on commodity desktop hardware. The encryption setup is straightforward, and the PRE flow is simple to understand.

The first transaction (`UseSnark`) verifies the Groth16 proof and commitment proof-of-knowledge via a withdraw validator. The Groth16 verification involves multiple pairing operations and public input accumulation, consuming most of the CPU budget. The commitment verification adds additional pairing checks. This separation allows the expensive SNARK verification to occur independently of the re-encryption logic.

The second transaction (`UseEncryption`) verifies limb compression (binding SNARK public inputs to G1 points), binding proofs, and pairing checks for the R5 term. The Schnorr and binding proofs are relatively cheap, but the pairing proofs remain expensive. By separating the SNARK verification into its own transaction, the re-encryption validation stays within budget.

The two-transaction design introduces additional complexity and a brief window where the status is `Pending`. However, this approach ensures the protocol remains feasible on-chain while providing the security guarantees of SNARK-verified witness creation. The constant on-chain storage (Section~4.1) also reduces datum size compared to storing all historical levels.

Concrete measurements.

- Compiled validator bytecode sizes: encryption 14,287 bytes, bidding 7,470 bytes, Groth16 verifier 2,857 bytes, genesis 2,072 bytes, reference 159 bytes
- Groth16 circuit: 1,084,616 constraints, 36 public inputs
- Proof generation: approximately 3 minutes on commodity desktop hardware (native Rust binary), with a proving key of 613 MiB
- The Groth16 structured reference string (SRS) is generated via a phase-2 ceremony implemented in the reference codebase; the trapdoor is assumed destroyed per the standard trusted-setup assumption

Table 3: Representative on-chain CPU budget consumption (Cardano mainnet limit: 10,000,000,000 CPU units per transaction)

Tx	Operation	CPU Units	% of Budget
Tx 1	Groth16 verification (gnark, with commitment PoK)	∼6,800,000,000–8,300,000,000	68–83%
Tx 1	Commitment PoK verification (standalone)	∼1,400,000,000	14%
Tx 2	R_5 pairing check	∼2,000,000,000	20%
Tx 2	Level validation (pairing + binding proof)	∼1,500,000,000	15%
Tx 2	Schnorr/binding proofs	∼290,000,000–1,300,000,000	3–13%
Tx 2	Limb compression verification	∼8,600,000–83,000,000	<1%

Memory consumption remains well below the 14,000,000-byte transaction limit ($<$3% in all cases). The two-transaction split is driven entirely by CPU budget: Groth16 verification alone consumes up to 83% of the per-transaction CPU limit, leaving insufficient headroom for the re-encryption pairing checks in the same transaction.

7 Conclusion

The PEACE protocol is a multi-use, unidirectional PRE for the Cardano blockchain with formal IND-CCA security guarantees (Appendix C). The security proof reduces to the PRE-DDH assumption (justified in the generic group model and implied by CDH in $\mathbb{G}_1$), Groth16 knowledge soundness, MiMC modeled as a random oracle, and standard symmetric-key assumptions. PEACE emphasizes correctness, composability, and an auditable trust boundary. Beyond implementing the Wang–Cao re-encryption core, the protocol addresses CCA vulnerabilities present in the original construction by contributing SNARK-verified witness binding, token-name ciphertext non-malleability, binding Σ-protocols for CCA decryption simulation, and constant on-chain storage independent of hop count. The two-transaction flow accommodates on-chain resource limits while maintaining security. We highlight the realities of the UTxO model, metadata leakage, and on-chain resource limits, and show how the design keeps cryptographic enforcement feasible while preserving a clear path toward stronger privacy and robustness. PEACE provides a concrete foundation for encrypted-asset markets on Cardano. It shows what is possible on-chain, what should remain off-chain, and how ownership can evolve across multiple hops while preserving decryption continuity for the rightful holder. The reference implementation is available at https://github.com/logical-mechanism/peace-protocol under the GNU General Public License v3.

A Appendix A - Data Structures

The pseudocode listings in this appendix use the following conventions: `scale(P, k)` denotes scalar multiplication $[k]P$; `combine(P, Q)` denotes group addition $P+Q$; `pair(P, Q)` denotes the bilinear pairing $e(P,Q)$; `mimc_hash(x, tag)` denotes $\mathsf{MiMC}(x\|tag)$; `rng()` returns a cryptographically secure random scalar; and `*` denotes scalar multiplication between integers. In mathematical expressions outside code listings, the standard $\cdot$ notation is used for multiplication.

```
pub type Register {
  // the generator, #<Bls12_381, G1> or #<Bls12_381, G2>
  generator: ByteArray,
  // the public value, #<Bls12_381, G1> or #<Bls12_381, G2>
  public_value: ByteArray,
}
```

Listing 1: The Register type

```
pub type BidDatum {
  owner_vkh: VerificationKeyHash,
  owner_g1: Register,
  pointer: AssetName,
  token: AssetName,
  locked_until: Int,
  new_price: Int,
}
```

Listing 2: The Bid datum type

```
pub type BidMintRedeemer {
  EntryBidMint(SchnorrProof)
  LeaveBidBurn(AssetName)
}
pub type BidSpendRedeemer {
  RemoveBid
  UseBid
  UpdateBidPrice(Int)
}
pub type SchnorrProof {
  z_b: ByteArray,
  g_r_b: ByteArray,
}
```

Listing 3: The Bid redeemer types

```
pub type EncryptionDatum {
  owner_vkh: VerificationKeyHash,
  owner_g1: Register,
  token: AssetName,
  half_level: HalfEncryptionLevel,
  full_level: Option<FullEncryptionLevel>,
  capsule: Capsule,
  status: Status,
  new_price: Int,
}
pub type Capsule {
  nonce: ByteArray,
  aad: ByteArray,
  ct: ByteArray,
}
pub type Status {
  Open
  Pending(GrothProof, GrothPublic, Int)
}
pub type HalfEncryptionLevel {
  r1b: ByteArray,
  r2_g1b: ByteArray,
  r4b: ByteArray,
}
pub type FullEncryptionLevel {
  r1b: ByteArray,
  r2_g1b: ByteArray,
  r2_g2b: ByteArray,
  r4b: ByteArray,
}
```

Listing 4: The Encryption datum type

```
pub type EncryptionMintRedeemer {
  EntryEncryptionMint(SchnorrProof, BindingProof)
  LeaveEncryptionBurn(AssetName)
}
pub type EncryptionSpendRedeemer {
  RemoveEncryption
  // R5 witness, R5, bid token, binding proof
  UseEncryption(ByteArray, ByteArray, AssetName, BindingProof)
  // groth proof and public come from groth_witness withdraw redeemer
  UseSnark
  CancelEncryption
  UpdateEncryptionPrice(Int)
}
pub type BindingProof {
  z_a_b: ByteArray,
  z_r_b: ByteArray,
  t_1_b: ByteArray,
  t_2_b: ByteArray,
}
```

Listing 5: The Encryption redeemer types

```
/// Example Usage:
///
/// input:
/// 1234567890abcdef1234567890abcdef1234567890abcdef1234567890abcdef#24
///
/// token_name:
/// 181234567890abcdef1234567890abcdef1234567890abcdef1234567890abcd
///
pub fn generate_token_name(inputs: List<Input>) -> AssetName {
  let input: Input = builtin.head_list(inputs)
  let id: TransactionId = input.output_reference.transaction_id
  let idx: Int = input.output_reference.output_index
  id |> bytearray.push(idx) |> bytearray.slice(0, 31)
}
```

Listing 6: Token name generation

```

message = "ThisIsASecretMessage."

# generate random data for the first encryption level
a0 = rng()
r0 = rng()
k0 = pair(scale(g, a0), h_0)

# alice as a register, sk is the secret key
alice = Register(sk)

# generate the r terms
r1b = scale(g, r0)
r2_g1b = scale(g, a0 + r0 * sk)

a = to_int(blake2b(r1b))
b = to_int(blake2b(r1b + r2_g1b + token_name))

c = combine(combine(scale(h_1, a), scale(h_2, b)), h_3)
r4b = scale(c, r0)

# encrypt the message
nonce, aad, ct = encrypt(r1b, k0, message)
```

Listing 7: Creating the Encrypted data and first encryption level

```
pub type HalfEncryptionLevel {
  r1b,
  r2_g1b,
  r4b,
}
pub type Capsule {
  nonce,
  aad,
  ct: ciphertext,
}
```

Listing 8: Encryption data format

```
assert pair(g, r4b) = pair(r1b, c)
```

Listing 9: First level validation

```
expected_k0 = pair(r2_g1b, h_0) / pair(r1b, scale(h_0, sk))
assert k0 == expected_k0
```

Listing 10: Alice can decrypt the key

```
pub type EncryptionDatum {
  owner_vkh,
  owner_g1,
  token: generate_token_name(inputs),
  half_level: HalfEncryptionLevel {
    r1b,
    r2_g1b,
    r4b,
  },
  full_level: None,
  capsule: Capsule {
    nonce,
    aad,
    ct: ciphertext,
  },
  status: Open,
  new_price: 0,
}
```

Listing 11: Full first level datum

```
pub type BidDatum {
  owner_vkh,
  owner_g1,
  pointer: generate_token_name(inputs),
  token,
  locked_until: now + 2 * minimum_bid_lock,
  new_price: bid_price,
}
```

Listing 12: Full Bid datum

```
a1 = rng()
r1 = rng()
k1 = pair(scale(g, a1), h_0)

hk = to_int(mimc_hash(k1, DOMAIN_TAG))

r1b = scale(g, r1)
r2_g1b = combine(scale(g, a1), scale(bob_public_value, r1))

a = to_int(blake2b(r1b))
b = to_int(blake2b(r1b + r2_g1b + token_name))
c = combine(scale(h_1, a), scale(h_2, b))
r4b = scale(c, r1)

r5b = combine(scale(p, hk), scale(invert(h_0), sk))
```

Listing 13: Generate the next level

```
// Bob's new half-level (current owner)
pub type HalfEncryptionLevel {
  r1b,
  r2_g1b,
  r4b,
}
// Alice's full-level (previous owner, stored for decryption)
pub type FullEncryptionLevel {
  r1b: alice.r1b,
  r2_g1b: alice.r2_g1b,
  r2_g2b: r5b,
  r4b: alice.r4b,
}
```

Listing 14: Bob's half-level and Alice's full-level

```
pub type EncryptionDatum {
  owner_vkh: bob.owner_vkh,
  owner_g1: bob.owner_g1,
  token,
  half_level: HalfEncryptionLevel {
    r1b,
    r2_g1b,
    r4b,
  },
  full_level: Some(FullEncryptionLevel {
    r1b: alice.r1b,
    r2_g1b: alice.r2_g1b,
    r2_g2b: r5b,
    r4b: alice.r4b,
  }),
  capsule: Capsule {
    nonce,
    aad,
    ct: ciphertext,
  },
  status: Open,
  new_price: bid.new_price,
}
```

Listing 15: Bob's encryption datum

```
assert pair(g, bob.r4b) = pair(bob.r1b, c)
assert pair(g, alice.r5b) * pair(alice.u, h_0) = pair(alice.witness, p)
```

Listing 16: Validate the re-encryption process

```
# Reconstruct encryption_levels by querying blockchain history
encryption_levels = query_chain_history(token, from_mint=True)

h0x = scale(h_0, sk)
shared = h0x

for entry in encryption_levels:
    r1 = entry.r1b

    if is_half_level(entry):
        r2 = pair(entry.r2_g1b, h_0)
    else:
        r2 = pair(entry.r2_g1b, h_0) * pair(r1, entry.r2_g2b)

    b = pair(r1, shared)
    key = mimc_hash(r2 / b, DOMAIN_TAG)
    k = to_int(key)
    shared = scale(p, k)

# final iteration's key decrypts the message
message = decrypt(r1, key, capsule.nonce, capsule.ct, capsule.aad)
```

Listing 17: Decrypting the secret message

```
pub fn fiat_shamir_heuristic(
  // compressed g element
  g_b: ByteArray,
  // compressed g^r element
  g_r_b: ByteArray,
  // compressed g^x element
  u_b: ByteArray,
) -> ByteArray {
  // concat g_b, g_r_b, u_b, and b together then hash the result
  schnorr_domain_tag
    |> bytearray.concat(g_b)
    |> bytearray.concat(g_r_b)
    |> bytearray.concat(u_b)
    |> crypto.blake2b_224()
  // Note: the binding proof variant replaces schnorr_domain_tag with
     BINDING|PROOF|v1|
}
```

Listing 18: Fiat–Shamir Transform Heuristic

B Appendix B - Proofs

Lemma B.1. *Correctness for Algorithm 1, a non-interactive Schnorr's Σ-protocol for the discrete logarithm relation.*

Proof. We start with $(\ g, u, a, z\ )$ where $g \in \mathbb{G}_1$, $u = [\delta]g \in \mathbb{G}_1$, $a = [r]g \in \mathbb{G}_1$, and $z = r + c \cdot \delta \in \mathbb{Z}_n$.

Use the Fiat–Shamir transform to generate a challenge value $c = R(g, u, a)$.

$[z]g = a + [c]u$

$[z]g = [r]g + [c][\delta]g$

$[z]g = [r + c \cdot \delta]g$

An honest `Register` can produce an a and z that will satisfy $[z]g = a + [c]u$ proving knowledge of the secret δ.

□

Lemma B.2. *Correctness for Algorithm 2, a non-interactive Σ-protocol for binding user data to encryption data.*

Proof. We start with $(\ g, u, t_1, t_2, z_a, z_r, r_1, \chi\ )$ where $g \in \mathbb{G}_1$, $u = [\delta]g \in \mathbb{G}_1$, $t_1 = [\rho]g \in \mathbb{G}_1$, $t_2 = [\alpha]g + [\rho]u \in \mathbb{G}_1$, $z_a = \alpha + c \cdot a \in \mathbb{Z}_n$, $z_r = \rho + c \cdot r \in \mathbb{Z}_n$, $r_1 = [r]g \in \mathbb{G}_1$, and $\chi = [a + r \cdot \delta]g \in \mathbb{G}_1$.

Use the Fiat–Shamir transform to generate a challenge value $c = R(g, u, t_1, t_2)$.

$[z_a]g + [z_r]u = t_2 + [c]\chi \wedge [z_r]g = t_1 + [c]r_1$

$[z_a]g + [z_r][\delta]g = [\alpha]g + [\rho][\delta]g + [c][a + r \cdot \delta]g \wedge [z_r]g = [\rho]g + [c][r]g$

$[z_a + z_r \cdot \delta]g = [\alpha + \rho \cdot \delta + c \cdot a + c \cdot r \cdot \delta]g \wedge [z_r]g = [\rho + c \cdot r]g$

$[z_a + z_r \cdot \delta]g = [\alpha + c \cdot a + \rho \cdot \delta + c \cdot r \cdot \delta]g \wedge [z_r]g = [\rho + c \cdot r]g$

An honest `Register` can produce a t_1, t_2, z_a, and z_r that will satisfy $z_a + z_r \cdot \delta = \alpha + c \cdot a + \rho \cdot \delta + c \cdot r \cdot \delta$ and $z_r = \rho + c \cdot r$ proving knowledge of the secret a and r while using u.

□

Lemma B.3. *Correctness for Algorithm 6, recursive decryption.*

Proof. Let us assume we have a single full encryption level and a half encryption level given by $(\ r_{1,a}, r_{2,a}\ )$ and $(\ r_{1,b}, r_{2,b}\ )$, respectively, where $r_{1,i} \in \mathbb{G}_1$, and $r_{2,i} \in \mathbb{G}_T$. The owner of the half level has secret $y \in \mathbb{Z}_n$ and the owner of the full level has secret $x \in \mathbb{Z}_n$. The point $h_0 = [s_0]p \in \mathbb{G}_2$ is a fixed public element.

Calculate κ_1 from the half level.

$\kappa_1 = \frac{r_{2,b}}{e(r_{1,b}, h_0^y)} = \frac{e(q^{a_1 + y \cdot r_1}, h_0)}{e(q^{r_1}, h_0^y)} = e(q^{a_1 + y \cdot r_1}, h_0) \cdot e(q^{r_1}, h_0^y)^{-1}$

$\kappa_1 = e(q^{a_1 + y \cdot r_1}, h_0) \cdot e(q^{-r_1 \cdot y}, h_0)$

$\kappa_1 = e(q^{a_1}, h_0)$

Calculate κ_0 from the full level.

$\kappa_0 = \frac{r_{2,a}}{e(r_{1,a}, p^{H(\kappa_1)})} = \frac{e(q^{a_0 + x \cdot r_0}, h_0) \cdot e(q^{r_0}, p^{H(\kappa_1)} \cdot h_0^{-x})}{e(q^{r_0}, p^{H(\kappa_1)})}$

$\kappa_0 = e(q^{a_0 + x \cdot r_0}, h_0) \cdot e(q^{r_0}, p^{H(\kappa_1)} \cdot h_0^{-x}) \cdot e(q^{-r_0 \cdot H(\kappa_1)}, p)$

$\kappa_0 = e(q^{s \cdot a_0 + s \cdot x \cdot r_0}, p) \cdot e(q^{r_0 \cdot H(\kappa_1) - r_0 \cdot s \cdot x}, p) \cdot e(q^{-r_0 \cdot H(\kappa_1)}, p)$

$\kappa_0 = e(q^{s \cdot a_0}, p) = e(q^{a_0}, h_0)$

Any user with κ_0 may decrypt the original message. When many full levels exist, the recursive relationship holds in general.

$\kappa_{i-1} = \frac{r_{2,i-1}}{e(r_{1,i-1}, p^{H(\kappa_i)})} = e(q^{a_{i-1}}, h_0)$

□

C Appendix C - Formal Security Analysis

This appendix establishes the IND-CCA security of the PEACE protocol via a sequence-of-games argument [24]. The proof accounts for the protocol's modifications to the Wang–Cao PRE scheme [6]: type-3 pairing over BLS12-381, token-name binding, SNARK-protected witness, and owner-mediated re-encryption.

C.1 Security Model

We define the IND-CCA security game for an owner-mediated proxy re-encryption scheme. The formalization draws on the frameworks of Canetti–Hohenberger [21] and Libert–Vergnaud [22], but omits the re-encryption oracle to accurately model PEACE's owner-mediated architecture (see justification below).

Definition C.1 (IND-CCA Game for Owner-Mediated PRE)**.** *Let λ be the security parameter. The game between a challenger $\mathcal{C}$ and a PPT adversary $\mathcal{A}$ proceeds as follows.*

Setup. *$\mathcal{C}$ runs the pairing parameter generation for BLS12-381, obtaining $(q, p, e, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, n)$, and publishes the fixed public parameters $(h_0, h_1, h_2, h_3) \in \mathbb{G}_2^4$ together with the Groth16 verification key vk.*

Oracles. *$\mathcal{A}$ has access to the following oracles:*

1. *$\mathcal{O}_{\mathsf{KeyGen}}(i)$: Generate a keypair (δ_i, u_i) where $\delta_i \xleftarrow{\$} \mathbb{Z}_n$ and $u_i = [\delta_i]q$. Return u_i and store δ_i.*
2. *$\mathcal{O}_{\mathsf{Corrupt}}(i)$: Return δ_i and mark user i as corrupted. The challenge user i^* may not be corrupted.*
3. *$\mathcal{O}_{\mathsf{Encrypt}}(i, m)$: Encrypt message m under user i's key at level 1. Return the ciphertext $\mathsf{CT} = (\mathsf{HalfLevel}, \mathsf{Capsule})$.*
4. *$\mathcal{O}_{\mathsf{Decrypt}}(i, \mathsf{CT})$: Decrypt CT under user i's key. Return the plaintext or $\perp$. Subject to the restriction that $\mathcal{A}$ may not query (i^*, CT^*) or any derivative of CT^* re-encrypted to a corrupted user. (Since no re-encryption oracle is provided in this game, the adversary cannot create such derivatives; this clause is vacuously satisfied but is included for consistency with the standard IND-PRE-CCA formulation in the PRE literature.)*

Challenge. *$\mathcal{A}$ submits (m_0, m_1, i^*) with $|m_0| = |m_1|$ and i^* uncorrupted. $\mathcal{C}$ samples $b \xleftarrow{\$} \{0, 1\}$, encrypts m_b under u_{i^*}, and returns CT^*.*

Output. *$\mathcal{A}$ outputs a guess b'. The advantage is $\mathsf{Adv}_{\mathcal{A}}^{\mathsf{IND\text{-}CCA}}(\lambda) = |\Pr[b' = b] - \frac{1}{2}|$.*

Why IND-CCA rather than IND-PRE-CCA. The standard IND-PRE-CCA game [21] includes a re-encryption oracle $\mathcal{O}_{\mathsf{ReEncrypt}}$ that models a proxy holding a standalone re-encryption key. PEACE uses owner-mediated re-encryption: the owner performs re-encryption directly using their secret δ, and no re-encryption key is ever published or delegated. The adversary cannot force the owner to re-encrypt; it can only observe re-encryptions the owner chooses to perform (which appear as on-chain transactions). Including $\mathcal{O}_{\mathsf{ReEncrypt}}$ would model a capability that does not exist in the protocol. The IND-CCA game accurately captures the PEACE threat model: the adversary has full access to encryption and decryption oracles, can corrupt non-challenge users, and observes all public on-chain data, but cannot compel the challenge user to re-encrypt. For non-challenge users $i \neq i^*$, the adversary can corrupt them and perform re-encryption itself using the extracted δ_i. Extending to full IND-PRE-CCA with a simulated re-encryption oracle for the challenge user

requires computing $[\delta_{i^*}]h_0 \in \mathbb{G}_2$ from $[\delta_{i^*}]q \in \mathbb{G}_1$, which is infeasible in a Type-3 pairing; this is an open problem for Wang–Cao style PRE on asymmetric pairings.

Additional modeling notes. (1) The on-chain verification checks (pairing equations, binding proofs, Schnorr proofs, Groth16 verification, limb compression) are public algorithms; $\mathcal{A}$ may run them on any candidate ciphertext. These constitute a well-formedness check, not a decryption oracle. (2) Each ciphertext is bound to a unique token name, so $\mathcal{A}$ cannot transplant a ciphertext across tokens.

C.2 Hardness Assumption

The core reduction requires a decisional assumption tailored to the PEACE ciphertext structure. In a Type-3 pairing, $\kappa = e([a]q, h_0)$ is efficiently computable from $[a]q \in \mathbb{G}_1$ and $h_0 \in \mathbb{G}_2$ via the public pairing map. Standard assumptions such as DBDH are not directly applicable because elements from both groups allow trivial pairing checks. We therefore define a custom assumption that captures the information-theoretic structure of PEACE ciphertexts.

Definition C.2 (PRE-DDH Assumption)**.** *Let $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ be a Type-3 bilinear pairing with generators $q \in \mathbb{G}_1$, $p \in \mathbb{G}_2$, and let $h_0 \in \mathbb{G}_2$ be a fixed public element with unknown discrete logarithm. For uniformly random $\delta, r, a \xleftarrow{\$} \mathbb{Z}_n$, the PRE-DDH problem is to distinguish:*

$$(q,\ [\delta]q,\ [r]q,\ [a + r\delta]q,\ p,\ [r]p,\ h_0,\ e([a]q, h_0))$$

$$\textit{from} \quad (q,\ [\delta]q,\ [r]q,\ [a + r\delta]q,\ p,\ [r]p,\ h_0,\ T)$$

where $T \xleftarrow{\$} \mathbb{G}_T$. The $\mathbb{G}_2$ element $[r]p$ enables the reducer to compute R_4 components that require the scalar r in $\mathbb{G}_2$. The advantage is $\mathsf{Adv}^{\mathsf{PRE\text{-}DDH}}(\lambda) = |\Pr[\mathcal{A}(\mathsf{real}) = 1] - \Pr[\mathcal{A}(\mathsf{random}) = 1]|$.

Lemma C.1 (PRE-DDH in the Generic Group Model)**.** *The PRE-DDH problem (Definition C.2) is hard in the generic group model for Type-3 pairings.*

Proof. In the GGM, the adversary's only operations are group operations in $\mathbb{G}_1$, $\mathbb{G}_2$, $\mathbb{G}_T$ and pairing evaluation e. Write $h_0 = [\eta]p$ for unknown η. The adversary's accessible $\mathbb{G}_1$ basis elements (as formal polynomials in the unknowns δ, r, a) are

$$\mathcal{B}_1 = \{1,\ \delta,\ r,\ a + r\delta\}$$

representing q, $[\delta]q$, $[r]q$, $[a+r\delta]q$ respectively, and the accessible $\mathbb{G}_2$ basis elements are

$$\mathcal{B}_2 = \{1,\ r,\ \eta\}$$

representing p, $[r]p$, $h_0 = [\eta]p$ respectively. Every computable $\mathbb{G}_T$ element is a pairing of a $\mathbb{G}_1$-linear combination with a $\mathbb{G}_2$-linear combination, so the accessible $\mathbb{G}_T$ monomials are all products $\alpha \cdot \beta$ with $\alpha \in \mathcal{B}_1$, $\beta \in \mathcal{B}_2$. Enumerating:

$\times$	1	r	η
1	1	r	η
δ	δ	δr	$\delta\eta$
r	r	r^2	$r\eta$
$a+r\delta$	$a+r\delta$	$ar+r^2\delta$	$a\eta+r\delta\eta$

The $\mathbb{Z}_n$-linear span of accessible $\mathbb{G}_T$ monomials is therefore span$\{1,\ r,\ r^2,\ \delta,\ \delta r,\ \delta\eta,\ \eta,\ r\eta,\ a{+}r\delta,\ ar{+}r^2\delta,\ a\eta{+}r\delta\eta\}$. The target is $a\eta = e([a]q, h_0)$. The monomial $a\eta$ appears only in the combination $a\eta + r\delta\eta$. To isolate $a\eta$, the adversary would need to subtract $r\delta\eta$. However, $r\delta\eta$ does not appear as an accessible monomial in isolation: although $\delta\eta$ and $r\eta$ are individually accessible, forming their product $\delta\eta \cdot r$ requires multiplication in $\mathbb{G}_T$, which is not a group operation (only the group law in the additive representation is available). Therefore, $a\eta$ is information-theoretically independent of the adversary's view, and PRE-DDH is hard in the GGM.

PRE-DDH is implied by CDH in $\mathbb{G}_1$: an adversary that solves CDH can compute $[r\delta]q$ from $[r]q$ and $[\delta]q$, recover $[a]q = [a{+}r\delta]q - [r\delta]q$, and then evaluate $e([a]q, h_0)$. □

Relation to standard assumptions. PRE-DDH is a falsifiable assumption that sits between DDH and CDH in strength. It is strictly weaker than CDH in $\mathbb{G}_1$ (CDH solves PRE-DDH, but not vice versa) and is independent of XDH (which concerns DDH in $\mathbb{G}_1$ without pairing access to the target). A tight reduction to XDH alone is complicated by the Type-3 coupling between $\mathbb{G}_1$ and $\mathbb{G}_2$ via the shared scalar r in the PEACE ciphertext structure (Appendix C.5, item 1). Achieving a tight reduction to XDH using techniques such as dual-system encryption or programmable hash proofs is left as future work. We stress that PRE-DDH is a custom assumption that has not been independently studied outside this work; custom assumptions carry inherent risk, as subtle structural properties may enable attacks not visible in the generic group model. A reduction to a well-studied assumption such as XDH or SXDH would materially strengthen confidence in the construction.

C.3 Main Theorem

Theorem C.1 (IND-CCA Security of PEACE)**.** *If the PRE-DDH assumption (Definition C.2) holds, MiMC behaves as a random oracle, Blake2b-224 is collision-resistant, hash-to-group is modeled as a random oracle, CDH holds in $\mathbb{G}_2$, Groth16 is knowledge-sound* [@groth-2016]*, AES-GCM is IND-CPA secure, and HKDF is a secure key derivation function in the random oracle model, then for any PPT adversary $\mathcal{A}$ there exist PPT reductions $\mathcal{B}_1, \ldots, \mathcal{B}_7$ such that*

$$\begin{aligned}\mathsf{Adv}^{\mathsf{IND\text{-}CCA}}_{\mathcal{A}}(\lambda) \le\ & \mathsf{Adv}^{\mathsf{PRE\text{-}DDH}}_{\mathcal{B}_1}(\lambda) + \mathsf{Adv}^{\mathsf{RO\text{-}MiMC}}_{\mathcal{B}_2}(\lambda) + \mathsf{Adv}^{\mathsf{CR\text{-}Blake2b}}_{\mathcal{B}_3}(\lambda) \\ & + \mathsf{Adv}^{\mathsf{CDH\text{-}}\mathbb{G}_2}_{\mathcal{B}_4}(\lambda) + \mathsf{Adv}^{\mathsf{KS\text{-}Groth16}}_{\mathcal{B}_5}(\lambda) + \mathsf{Adv}^{\mathsf{IND\text{-}CPA\text{-}AES}}_{\mathcal{B}_6}(\lambda) + \mathsf{Adv}^{\mathsf{PRF\text{-}HKDF}}_{\mathcal{B}_7}(\lambda)\end{aligned} \tag{C.1}$$

where the PRE-DDH term absorbs an $O(Q_D \cdot Q_H)$ tightness loss from forking-lemma extraction in the decryption simulation, with Q_D the number of challenge-user decryption queries and Q_H the number of random oracle queries.

Proof. By a sequence of seven games. Let S_i denote the event that $\mathcal{A}$ wins in Game i.

Game 0. The real IND-CCA game as defined in Definition C.1. By definition, $\mathsf{Adv}_{\mathcal{A}} = |\Pr[S_0] - \frac{1}{2}|$.

Game 1 (reject malformed decryption queries). Modify $\mathcal{O}_{\mathsf{Decrypt}}$ to re-run the on-chain verification checks before decrypting: the pairing equation $e(q, R_4) = e(R_1, h_1^{H(R_1)} \cdot h_2^{H(R_1 \| R_2 \| \mathsf{token})} \cdot h_3)$ for level 1 (or the analogous equation without h_3 for level k), the R5 check $e(q, R_5) \cdot e(u, h_0) = e(W, p)$, binding proof verification, and Schnorr proof verification. Return $\perp$ on any failure.

Claim C.1. $\Pr[S_1] = \Pr[S_0]$.

This is a syntactic change. In the real protocol, any ciphertext accepted by the on-chain validator is well-formed, and $\mathcal{O}_{\mathsf{Decrypt}}$ already requires well-formedness implicitly. No advantage is gained or lost.

Game 2 (SNARK knowledge extraction). Replace the Groth16 prover with a simulator. By the knowledge-soundness of Groth16 with the gnark commitment extension, for any valid proof π that passes the pairing verification $e(A, B) \cdot e(vk_x, -\gamma) \cdot e(C, -\delta) \cdot e(\alpha, -\beta) = 1$ and the Pedersen commitment proof-of-knowledge, there exists an extractor $\mathcal{E}$ that recovers the witness (a, r) satisfying $w_0 = [\mathsf{MiMC}(\kappa \| \mathsf{DomainTag})]q$ and $w_1 = [a]q + [r]v$, where $\kappa = e([a]q, h_0)$.

Claim C.2. $|\Pr[S_2] - \Pr[S_1]| \leq \mathsf{Adv}^{\mathsf{KS\text{-}Groth16}}_{\mathcal{B}_5}(\lambda)$.

The commitment proof-of-knowledge provides additional binding: the Pedersen commitment PoK ensures the commitment wire is correctly derived from the committed public inputs, preventing witness substitution attacks.

Game 3 (random oracle model). Henceforth, the proof works in the random oracle model (ROM). The Blake2b-224 hash used in the Fiat–Shamir transforms is modeled as a random oracle, with the Schnorr Σ-protocol using domain tag `SCHNORR|PROOF|v1|` and the binding Σ-protocol using `BINDING|PROOF|v1|`. Domain separation between the two proof types prevents cross-protocol attacks.

Claim C.3. *Games 2 and 3 are identical in the ROM.*

This is a modeling assumption, not a computational transition: the Fiat–Shamir transform is secure in the ROM by the forking lemma, and the special soundness of both Σ-protocols is preserved. Instantiation with Blake2b-224 is a heuristic whose confidence rests on the collision resistance of the Blake2 family. The $\mathsf{Adv}^{\mathsf{CR\text{-}Blake2b}}$ term in the main bound is attributed to Game 4, where hash collision resistance is concretely used for R_4 binding. The binding proof (Algorithm 2) proves knowledge of (a, r) such that $\chi = [a]q + [r]u$ and $r_1 = [r]q$, which is exactly the structure of the ciphertext's R_2 and R_1 components.

Game 4 (ciphertext non-malleability via R4 binding). Show that the R_4 component binds the ciphertext components together. The verification equation $e(q, R_4) = e(R_1, h_1^{H(R_1)} \cdot h_2^{H(R_1 \| R_2 \| \mathsf{token})} \cdot h_3)$ ties $(R_1, R_2, \mathsf{token})$ through hash-derived exponents on fixed public $\mathbb{G}_2$ points.

Claim C.4. $|\Pr[S_4] - \Pr[S_3]| \leq \mathsf{Adv}^{\mathsf{CR\text{-}Blake2b}}_{\mathcal{B}_3}(\lambda) + \mathsf{Adv}^{\mathsf{CDH\text{-}}\mathbb{G}_2}_{\mathcal{B}_4}(\lambda)$.

Suppose $\mathcal{A}$ produces a modified ciphertext (R_1', R_2', R_4') that passes the pairing check with a different (R_1, R_2) or a different token name. Under hash collision resistance ($\mathsf{Adv}^{\mathsf{CR\text{-}Blake2b}}$), if $(R_1', R_2', \mathsf{token}')$ differs from the original, the hash outputs $c_1' = H(R_1')$ and $c_2' = H(R_1' \| R_2' \| \mathsf{token}')$ are fresh random oracle values, producing a genuinely new $c' = [c_1']h_1 + [c_2']h_2 + h_3 \in \mathbb{G}_2$. The adversary must then produce $R_4' = [r']c'$ satisfying $e(q, R_4') = e(R_1', c')$, which means R_4' and $R_1' = [r']q$ share the same scalar r'. This is a cross-group Diffie–Hellman extraction: computing $[r']c' \in \mathbb{G}_2$ from $[r']q \in \mathbb{G}_1$ and $c' \in \mathbb{G}_2$ without knowing r' as a scalar. In a Type-3 pairing with no efficiently computable homomorphism from $\mathbb{G}_1$ to $\mathbb{G}_2$, this is infeasible in the GGM. We name this $\mathsf{CDH\text{-}}\mathbb{G}_2$ as the closest standard assumption, noting that the actual hardness is a cross-group variant that holds in the GGM for Type-3 pairings.

PEACE-specific note: The inclusion of the token name in the hash input (absent in the original Wang–Cao construction) prevents cross-NFT transplant attacks, where an adversary copies a valid ciphertext from one token to another.

Game 5 (PRE-DDH core). In the challenge ciphertext, replace $\kappa^* = e([a^*]q, h_0)$ with a uniformly random element $\kappa_{\mathsf{rand}} \in \mathbb{G}_T$.

Claim C.5. $|\Pr[S_5] - \Pr[S_4]| \leq \mathsf{Adv}^{\mathsf{PRE\text{-}DDH}}_{\mathcal{B}_1}(\lambda) + \mathsf{Adv}^{\mathsf{RO\text{-}MiMC}}_{\mathcal{B}_2}(\lambda)$.

Reduction. Given a PRE-DDH instance $(q,\ [\delta]q,\ [r]q,\ [a + r\delta]q,\ p,\ [r]p,\ h_0,\ T)$ where T is either $e([a]q, h_0)$ (real) or uniform in $\mathbb{G}_T$ (random), the reducer $\mathcal{B}_1$ simulates the IND-CCA game as follows:

- Set the challenge user's public key: $u_{i^*} = [\delta]q$.
- Set the challenge ciphertext components: $R_1^* = [r]q$, $R_2^* = [a + r\delta]q$.
- Set the pairing secret: $\kappa^* = T$.
- Derive $hk^* = \mathsf{MiMC}(\kappa^* \| \mathsf{DomainTag})$ and $\mathsf{DEK}^* = \mathsf{HKDF}(\kappa^* \| R_1^*)$, encrypt m_b.

If $T = e([a]q, h_0)$, the challenge ciphertext is correctly distributed. If T is random, κ^* is uniform in $\mathbb{G}_T$, and the DEK is independent of the plaintext.

Simulating R_4^*. The reducer does not know r as a scalar, but the PRE-DDH instance provides $[r]p \in \mathbb{G}_2$. During setup, the reducer chooses $h_1 = [\eta_1]p$, $h_2 = [\eta_2]p$, $h_3 = [\eta_3]p$ with known scalars η_1, η_2, η_3 (modeling hash-to-group as a random oracle). The reducer computes $c_1 = H(R_1^*)$ and $c_2 = H(R_1^* \| R_2^* \| \mathsf{token})$ from the random oracle, yielding the known scalar $s = c_1\eta_1 + c_2\eta_2 + \eta_3$. Then $R_4^* = [s]([r]p) = [rs]p$, which equals $[r](h_1^{c_1} \cdot h_2^{c_2} \cdot h_3)$ as required. For all other ciphertexts, honest provers who know their own r can compute R_4 correctly.

Simulating proofs. The challenge ciphertext's binding proof and Schnorr proof are simulated via Fiat–Shamir programming in the random oracle model (enabled by Game 3). The reducer programs the random oracle to produce challenges that make simulated transcripts accepting, without knowledge of the witness scalars (a, r). The SNARK proof is simulated using the Groth16 simulator (enabled by Game 2).

MiMC as random oracle. Once κ^* is uniform in $\mathbb{G}_T$, the MiMC random oracle outputs $hk = \mathsf{MiMC}(\kappa^* \| \mathsf{DomainTag})$ uniformly in $\mathbb{Z}_n$, and the DEK derived via HKDF becomes independent of the plaintext.

Simulating R_5. The challenge ciphertext at level-1 has no R_5 component (R_5 is created only during re-encryption). R_5 simulation applies only to re-encrypted ciphertexts observed by $\mathcal{A}$. Since PEACE is owner-mediated, these are re-encryptions that non-challenge users perform voluntarily; the reducer knows their secret keys and computes $R_5 = [H(\kappa)]p + [\delta](-h_0)$ directly. For the challenge user, no re-encryption oracle exists (IND-CCA model), so no R_5 simulation under i^* is needed.

Simulating decryption. The reducer does not know δ_{i^*}, so it cannot decrypt ciphertexts under the challenge user directly. Instead, it exploits the extraction machinery established in Games 1–3. For any decryption query CT under user i^* (that is not the challenge ciphertext or a derivative):

1. Game 1 mandates that CT passes all on-chain verification checks, including a valid binding proof; if any check fails, return $\perp$. This guarantees that every query reaching step 2 contains an extractable binding proof.
2. Game 3 provides binding proof extraction in the ROM via the forking lemma: the reducer forks

the adversary at the random oracle query that produced the Fiat–Shamir challenge c, supplies a fresh challenge c', and obtains a second accepting transcript. By special soundness of the binding Σ-protocol, the two transcripts yield (a, r) such that $R_1 = [r]q$ and $R_2 = [a]q + [r]u_{i^*}$.

3. With extracted a, compute $\kappa = e([a]q, h_0)$ directly (the pairing is efficiently computable given $[a]q \in \mathbb{G}_1$ and $h_0 \in \mathbb{G}_2$), derive the DEK via HKDF, and decrypt.

Tightness of forking. The reducer handles Q_D decryption queries under i^* using the general forking lemma: the reducer records all random oracle queries and, for each decryption query requiring extraction, forks the adversary at the relevant Fiat–Shamir query point to obtain a second transcript with a different challenge. The general forking lemma bounds the total extraction cost without nesting: during a fork replay, the adversary may submit new decryption queries, but the reducer answers them from the ROM table and the recorded transcript (which is consistent across the fork). No recursive forking occurs because extraction depends only on the ROM state, not on previously extracted values. Concretely, during a fork replay the adversary may submit new decryption queries not seen in the original run; the reducer answers these using the same random oracle table it controls, so witness extraction for any new query is a table lookup followed by a single additional fork at the new query's Fiat–Shamir point, not a nested fork within the existing replay. This "flat" forking structure is the key insight of the general forking lemma [@bellare-neven-2006]: each extraction is independent and touches a disjoint random oracle query point, so the total cost is additive rather than multiplicative. The total tightness loss is $O(Q_D \cdot Q_H)$, absorbed into the PRE-DDH term in the main bound. For non-challenge users, the reducer knows their secret keys and decrypts normally.

Game 6 (replace DEK). Now that κ^* is random in $\mathbb{G}_T$, replace the DEK $k = \mathsf{HKDF}(\kappa^* \| R_1^*)$ with a uniformly random key k_{rand}.

Claim C.6. $|\Pr[S_6] - \Pr[S_5]| \leq \mathsf{Adv}^{\mathsf{PRF\text{-}HKDF}}_{\mathcal{B}_7}(\lambda)$.

By the extraction security of HKDF in the random oracle model: when the source material κ^* has sufficient min-entropy (it is uniform in $\mathbb{G}_T$, providing $\log_2 n$ bits), the HKDF output is computationally indistinguishable from uniform.

Game 7 (replace AES-GCM ciphertext). With a uniformly random DEK, replace the AES-GCM encryption of m_b with an encryption of a fixed message $0^{|m_b|}$.

Claim C.7. $|\Pr[S_7] - \Pr[S_6]| \leq \mathsf{Adv}^{\mathsf{IND\text{-}CPA\text{-}AES}}_{\mathcal{B}_6}(\lambda)$.

By the IND-CPA security of AES-GCM under a random key. In Game 7, the adversary's view is completely independent of the challenge bit b, so $\Pr[S_7] = \frac{1}{2}$.

Collecting the bounds across all transitions yields the theorem statement (C.1). □

C.4 Key Lemmas

Lemma C.2 (Binding proof soundness)**.** *The binding Σ-protocol (Algorithm 2) is a proof of knowledge for the relation*

$$\mathcal{R} = \{((g, u, r_1, \chi), (a, r)) : u = [\delta]g,\ r_1 = [r]g,\ \chi = [a]g + [r]u\}$$

with knowledge error 2^{-224} (from the Blake2b-224 challenge space) in the random oracle model.

Proof sketch. Standard special-soundness argument for parallel composition of Schnorr protocols. Given two accepting transcripts (t_1, t_2, c, z_a, z_r) and $(t_1, t_2, c', z'_a, z'_r)$ with $c \neq c'$, we extract:

$$r = \frac{z_r - z'_r}{c - c'}, \quad a = \frac{z_a - z'_a}{c - c'}$$

The Fiat–Shamir transform with domain tag `BINDING|PROOF|v1|` preserves proof-of-knowledge in the ROM. □

Lemma C.3 (R5 unforgeability)**.** *Given the pairing check $e(q, R_5) \cdot e(u, h_0) = e(W, p)$ and the SNARK proving $W = [\mathsf{MiMC}(e([a]q, h_0) \| \mathsf{DomainTag})]q$, producing a valid (R_5, W) pair requires knowledge of δ (the owner's secret key).*

Proof sketch. From the pairing equation: $R_5 = [H(\kappa)]p - [\delta]h_0$ (writing additively in $\mathbb{G}_2$). The SNARK binds $H(\kappa)$ to the actual pairing secret $\kappa = e([a]q, h_0)$ via knowledge soundness, so the adversary cannot choose $H(\kappa)$ freely. Producing a valid R_5 without knowing δ then requires solving the discrete logarithm problem in $\mathbb{G}_2$ with respect to h_0, given the fixed value $[H(\kappa)]p$. □

Lemma C.4 (Limb compression binding)**.** *The limb compression verification correctly binds the SNARK public inputs (36 integers representing the affine coordinates of three $\mathbb{G}_1$ points as 64-bit limbs) to the compressed $\mathbb{G}_1$ points stored in the datum. Any mismatch is detected.*

Proof sketch. The function reconstructs compressed $\mathbb{G}_1$ points from the little-endian 64-bit limbs and compares byte-for-byte against the expected compressed representations. Since BLS12-381 $\mathbb{G}_1$ compression (Zcash serialization [@ZcashProtocolSpec2022NU5]) is injective — each valid point has exactly one canonical compressed representation — this check is both complete (valid limbs always pass) and sound (invalid limbs always fail). □

Lemma C.5 (Multi-hop security degradation)**.** *For a chain of n re-encryption hops, the IND-CCA advantage degrades at most linearly in n:*

$$\mathsf{Adv}^{n\text{-}hop} \leq n \cdot \mathsf{Adv}^{\mathsf{PRE\text{-}DDH}} + \mathsf{negl}(\lambda)$$

Proof sketch. Each hop i introduces fresh random (a_i, r_i) and a fresh $\kappa_i = e([a_i]q, h_0)$. The recursive decryption structure (Lemma B.3) means that κ_i reveals κ_{i-1} via the R_5 chain: $R_{5,i-1} = [H(\kappa_i)]p + [\delta_{i-1}](-h_0)$ links consecutive hops. However, this backward dependency does not compromise forward security. To see why: $\kappa_i = e([a_i]q, h_0)$ depends only on the fresh secret a_i, which is independent of all prior secrets $(a_0, \ldots, a_{i-1})$ and all prior keys $(\delta_0, \ldots, \delta_{i-1})$. The R_5 chain creates algebraic dependencies in the backward direction (decryption), but in the forward direction, each hop's PRE-DDH instance $([\delta_i]q, [r_i]q, [a_i + r_i\delta_i]q, h_0)$ uses fresh (a_i, r_i) that are information-theoretically independent of all prior hops' secrets. A standard hybrid argument over the n hops gives the linear bound: hybrid j replaces κ_j with a random $\mathbb{G}_T$ element, and the transition from hybrid j to $j{+}1$ reduces to a single PRE-DDH instance. When κ_j is replaced with random, the R_5 chain's algebraic structure changes for all hops $< j$: specifically, $R_{5,j-1}$ now contains $H(\kappa_{j,\mathsf{rand}})$ instead of $H(\kappa_j)$. However, this does not affect the PRE-DDH instance at hop $j{+}1$, because hop $j{+}1$'s secret (a_{j+1}, r_{j+1}) is fresh and independent of the R_5 chain below it. The change in $R_{5,j-1}$ only affects decryption of hops $\leq j{-}1$, not the security of hops $> j$. Unidirectionality ensures that knowing δ_j (Bob's secret) does not reveal δ_i (Alice's secret), since only the R_5 chain (which requires δ_i to construct) links the two. □

C.5 PEACE-Specific Differences from Wang–Cao

The following modifications distinguish PEACE from the original Wang–Cao construction and affect the security argument:

1. **Type-3 pairing and PRE-DDH.** Wang–Cao is stated for a symmetric (Type-1) pairing where DBDH is a natural hardness assumption. PEACE uses BLS12-381 (Type-3), where no efficiently computable isomorphism exists between $\mathbb{G}_1$ and $\mathbb{G}_2$. This prevents certain self-pairing attacks but introduces a structural coupling: the ciphertext uses the same scalar r in both $\mathbb{G}_1$ ($R_1 = [r]q$) and $\mathbb{G}_2$ ($R_4 = [r]c$). Standard assumptions like DBDH are not directly applicable because cross-group elements allow trivial pairing checks. The proof uses the custom PRE-DDH assumption (Definition~C.2), which holds in the generic group model and is implied by CDH in $\mathbb{G}_1$. A tight reduction to XDH alone, using techniques such as dual-system encryption or programmable hash proofs, remains an open problem.

2. **Token-name binding in R_4.** The hash inputs for the R_4 pairing check include the `token_name`, binding each ciphertext to a specific on-chain NFT. This strengthens non-malleability (Game~4) and is absent in Wang–Cao.

3. **SNARK-protected witness.** The re-encryption witness $W = q^{H(\kappa)}$ is bound to κ via a Groth16 proof rather than an algebraic argument. This replaces a direct algebraic reduction with an appeal to knowledge soundness (Game~2).

4. **Owner-mediated re-encryption and IND-CCA model.** The delegator performs re-encryption directly; no standalone re-encryption key is published or delegated. This means the standard IND-PRE-CCA re-encryption oracle $\mathcal{O}_{\mathsf{ReEncrypt}}$ does not model a real protocol capability, so the proof uses the IND-CCA game instead. Simulating $\mathcal{O}_{\mathsf{ReEncrypt}}$ for the challenge user would require computing $[\delta_{i^*}]h_0 \in \mathbb{G}_2$ from $[\delta_{i^*}]q \in \mathbb{G}_1$, which is infeasible in a Type-3 pairing. Extending to full IND-PRE-CCA for owner-mediated PRE on asymmetric pairings remains an open problem.

5. **Two-transaction flow and Pending state.** The SNARK submission and re-encryption completion occur in separate on-chain transactions. During the Pending state, the SNARK public inputs (w_0, w_1) and proof are visible on-chain. We argue this does not leak information beyond the completed re-encryption: the public inputs are $\mathbb{G}_1$ points that are already published in the next half-level after completion, and the TTL is a timing parameter.

6. **MiMC for $\mathbb{G}_T$-to-scalar hashing.** MiMC is chosen for arithmetic circuit efficiency, not cryptographic conservatism (see Discussion below). The proof models MiMC as a random oracle (Game~5), which is stronger than collision resistance but is the standard treatment for algebraic hash functions in SNARK-based protocols. The random oracle assumption ensures that $\mathsf{MiMC}(\kappa \| \mathsf{DomainTag})$ is uniformly distributed when κ is uniform in $\mathbb{G}_T$. MiMC may be replaced with Poseidon as a drop-in substitution in the SNARK circuit without affecting the rest of the proof.

7. **h_3 term in level-1 vs. level-k.** Level-1 ciphertexts include h_3 in the R_4 pairing check; level-k ciphertexts do not. This provides domain separation between first-level and re-encrypted ciphertexts, preventing downgrade attacks where an adversary attempts to present a level-k ciphertext as a level-1 ciphertext (or vice versa).

C.6 Discussion and Limitations

MiMC random oracle assumption. MiMC is chosen for arithmetic circuit efficiency, not cryptographic conservatism. It has low multiplicative complexity (a single cubing per round), which makes it orders of magnitude cheaper inside a Groth16 circuit than SHA-256 or Blake2b, but its algebraic structure admits dedicated interpolation and GCD attacks that do not apply to non-algebraic hashes. The random oracle model (Game~5) is therefore a stronger-than-usual assumption for MiMC: it requires not just collision resistance but output indistinguishability, over a function whose round structure was designed for circuit efficiency rather than maximal security margin. This is a pragmatic engineering choice with an explicit upgrade path: the SNARK circuit's hash function can be replaced with Poseidon (or a future algebraic hash with stronger analysis) without affecting the remaining proof structure or the on-chain verifier.

Groth16 trusted setup. Knowledge soundness of Groth16 depends on the structured reference string (SRS) being honestly generated, i.e., the trapdoor ("toxic waste") is destroyed. The reference implementation includes a phase-2 ceremony tool for SRS generation; in a production deployment, this ceremony should be executed by multiple independent parties to distribute trust.

Random oracle model. The Fiat–Shamir transforms for the Schnorr and binding Σ-protocols (Game~3) are proven secure in the ROM. The proof does not claim security in the standard model for these components. Instantiation with Blake2b-224 is a heuristic, albeit one supported by the extensive cryptanalysis of the Blake2 family.

AES-GCM nonce management. The IND-CPA security of AES-GCM (Game~7) requires that nonces are never reused under the same key. Nonce management is an operational requirement handled by the off-chain implementation, not by the on-chain protocol.

Multi-hop linear degradation. Lemma~C.5 shows linear degradation in the number of hops. For practical hop counts (tens to low hundreds), this remains well within security margins for 128-bit security on BLS12-381.

Bibliography

[1] J. Stone, "Stuff.io whitepaper 1.0." Accessed: Oct. 25, 2025. [Online]. Available: https://book-io.medium.com/stuff-io-whitepaper-1-0-9529db7cdeaf

[2] M. Mambo and E. Okamoto, "Proxy cryptosystems: Delegation of the power to decrypt ciphertexts," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E80–A, no. 1, pp. 54–63, 1997, Available: https://search.ieice.org/bin/summary.php?id=e80-a_1_54

[3] M. Blaze, G. Bleumer, and M. Strauss, "Divertible protocols and atomic proxy cryptography," in *EUROCRYPT 1998*, Springer, 1998. doi: 10.1007/BFb0054122.

[4] G. Ateniese, K. Fu, M. Green, and S. Hohenberger, "Improved proxy re-encryption schemes with applications to secure distributed storage," in *NDSS 2005*, 2005. Available: https://www.ndss-symposium.org/ndss2005/improved-proxy-re-encryption-schemes-applications-secure-distributed-storage/

[5] IronCore Labs, *Recrypt (rust): Transform/proxy re-encryption library*. Accessed: Oct. 25, 2025. [Online]. Available: https://github.com/IronCoreLabs/recrypt-rs

[6] H. Wang and Z. Cao, "A fully secure unidirectional and multi-use proxy re-encryption scheme." Poster, 16th ACM Conference on Computer and Communications Security (CCS 2009), Chicago, IL, USA, Nov. 9–13, 2009, 2009. Available: https://www.sigsac.org/ccs/CCS2009/pd/abstract_16.pdf

[7] *IEEE standard specifications for public-key cryptography-amendment 1: Additional techniques*. IEEE, 2004.

[8] *Advanced encryption standard (AES)*. NIST, 2001.

[9] S. Bowe, "BLS12-381: New zk-SNARK elliptic curve construction." Accessed: Oct. 25, 2025. [Online]. Available: https://electriccoin.co/blog/new-snark-curve/

[10] aiken-lang contributors, *Aiken: A modern smart contract platform for cardano*. (2025). Accessed: Dec. 16, 2025. [Online]. Available: https://github.com/aiken-lang/aiken

[11] N. El Mrabet and M. Joye, Eds., *Guide to pairing-based cryptography*. in Chapman & hall/CRC cryptography and network security. New York: Chapman; Hall/CRC, 2017, p. 420. doi: 10.1201/9781315370170.

[12] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Advances in cryptology – CRYPTO '86*, in Lecture notes in computer science, vol. 263. Springer, 1986, pp. 186–194. doi: 10.1007/3-540-47721-7_12.

[13] S. Josefsson and I. Liusvaara, *Edwards-curve digital signature algorithm (EdDSA)*. IETF, 2017. Available: https://www.rfc-editor.org/rfc/rfc8032

[14] J.-P. Aumasson, S. Neves, Z. Wilcox-O'Hearn, and C. Winnerlein, *The BLAKE2 cryptographic hash and message authentication code (MAC)*. IETF, 2015. Available: https://www.rfc-editor.org/rfc/rfc7693

[15] D. Hopwood, S. Bowe, T. Hornby, and N. Wilcox, "Zcash protocol specification," Electric Coin Company, Version 2022.3.8 [NU5], Sep. 2022. Available: https://resources.cryptocompare.com/asset-management/102/1672746915982.pdf

[16] J. Thaler, "Proofs, arguments, and zero-knowledge," *Foundations and Trends in Privacy and Security*, vol. 4, no. 2–4, pp. 117–660, 2022, doi: 10.1561/3300000030.

[17] C.-P. Schnorr, "Efficient signature generation by smart cards," *Journal of Cryptology*, vol. 4, no. 3, pp. 161–174, 1991, doi: 10.1007/BF00196725.

[18] A. J. Menezes, *Elliptic curve public key cryptosystems*, vol. 234. in The springer international series in engineering and computer science, vol. 234. Boston, MA: Kluwer Academic Publishers, 1993. doi: 10.1007/978-1-4615-3198-2.

[19] H. Krawczyk, "Cryptographic extraction and key derivation: The HKDF scheme." Cryptology ePrint Archive, Paper 2010/264, 2010. Available: https://eprint.iacr.org/2010/264

[20] G. Ateniese, K. Fu, M. Green, and S. Hohenberger, "Improved proxy re-encryption schemes with applications to secure distributed storage," *ACM Transactions on Information and System Security*, vol. 9, no. 1, pp. 1–30, 2006, doi: 10.1145/1127345.1127346.

[21] R. Canetti and S. Hohenberger, "Chosen-ciphertext secure proxy re-encryption," in *ACM conference on computer and communications security (CCS 2007)*, ACM, 2007, pp. 185–194. doi: 10.1145/1315245.1315269.

[22] B. Libert and D. Vergnaud, "Unidirectional chosen-ciphertext secure proxy re-encryption," in *Public key cryptography – PKC 2008*, in Lecture notes in computer science, vol. 4939. Springer, 2008, pp. 360–379. doi: 10.1007/978-3-540-78440-1_21.

[23] A. Konrad and T. Vellekoop, "CIP-68: Datum metadata standard." https://cips.cardano.org/cip/CIP-68, 2022.

[24] V. Shoup, "Sequences of games: A tool for taming complexity in security proofs." Cryptology ePrint Archive, Paper 2004/332, 2004. Available: https://eprint.iacr.org/2004/332